SORTING AND SEARCHING

6.1 SORTING

Sorting is the process of arranging the data or information in some logical order. That may be ascending or descending or dictionary order (for Characters).

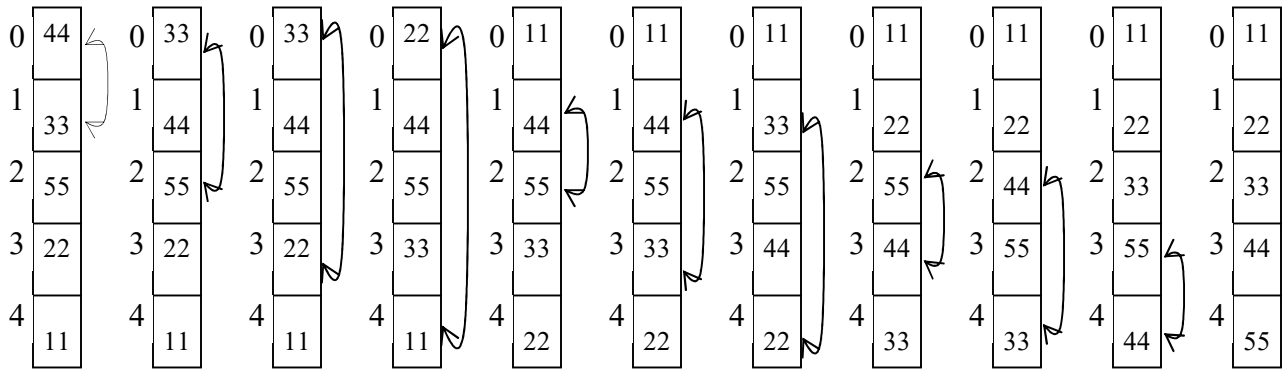
The various sorting techniques that are available are:

1. Selection Sort
2. Bubble Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Radix Sort
7. Heap Sort
8. Address Calculation Sort (i.e. Using Hashing)

.

6.1.1 SELECTION SORT

In selection sort technique, the smallest element in the list is found out by comparing the first element with second, third and so on up to last position. At the end of first pass, the smallest element of the list will be at the 1st position. Now the second smallest element is found out by comparing second element with third, fourth and so on up to last position. At the end second pass, the 2nd smallest element will be at the 2nd position. This process continues for rest of the elements so as to get the list sorted. We require (N-1) number of passes to sort a list of N elements.

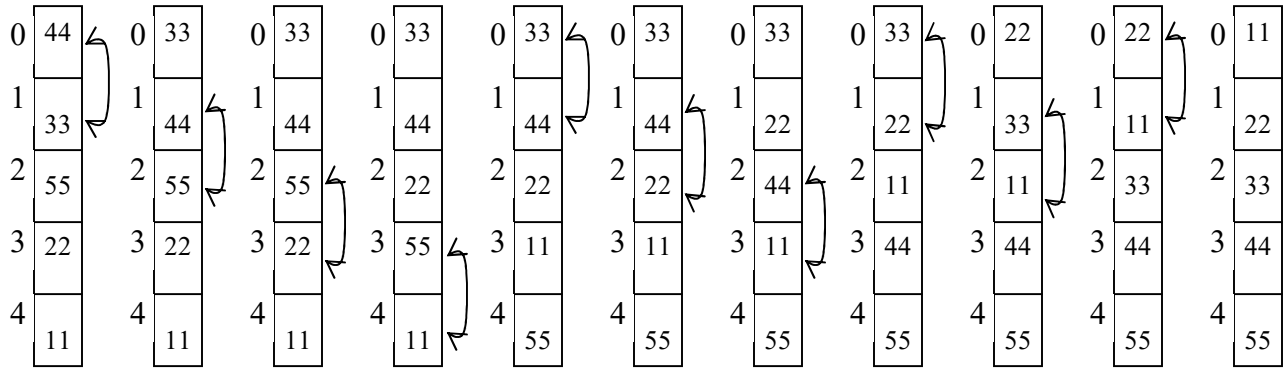


SELECTION_SORT (A [N])

```
Step 1: Let I, J, TEMP
Step 2:        Repeat for I: =0 to N – 1 increasing by 1
        Step 2.1: Repeat for J: =I+1 to N increasing by 1
                Step 2.1.1: if A [I] > A [J], Then
                                TEMP: =A [I]
                                A [I]:=A [J]
                                A [J]:=TEMP
                            [End of If]
                [End of For]
        [End of For]
Step 3: Exit
```

6.1.2 BUBBLE SORT

In bubble sort, each element is compared with its adjacent element. If the first element is larger than the second one then the position of the elements is interchanged, otherwise it is not changed. Then next element is compared with its adjacent element and the same process is repeated for all the elements in the array. At the end of Pass-1, the largest element in the list will occupy the last position, at the end of Pass-2, the second largest element will occupy the 2nd last position and so on.



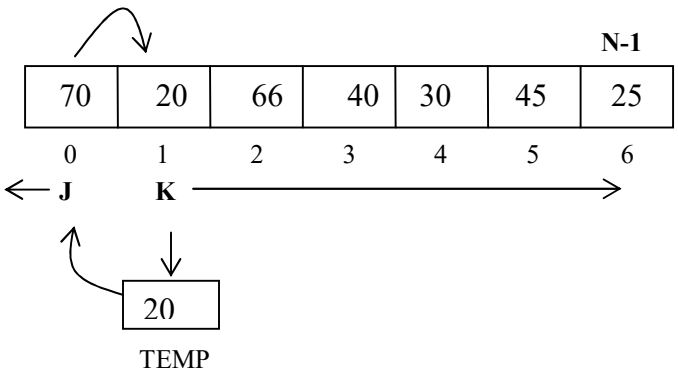
```
BUBBLE_SORT (A [N])

Step 1: Let I, K, TEMP
Step 2:        Repeat for K: =N-2 to 0 decreasing by 1
        Step 2.1: Repeat for I: =0 to K increasing by 1
                Step 2.1.1: If A [I] >A [I +1], then
                                TEMP: =A [I]
                                A [I]:=A [I+1]
                                A [I+1]:=TEMP
                            [End of If]
                [End of For]
        [End of For]
Step 3:        Exit
```

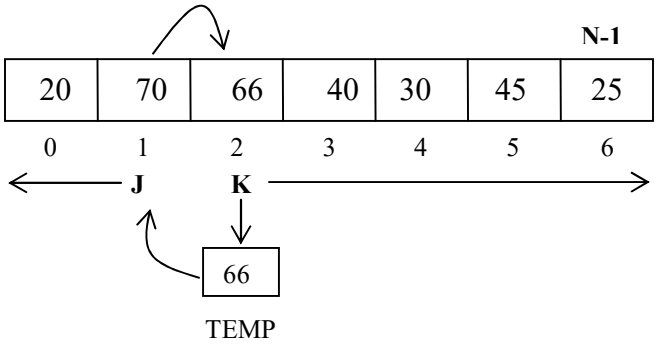
6.1.3 INSERTION SORT

Insertion sort is very much similar in the way of arranging the cards by a card player. The card player picks up the cards and inserts them in the proper position such that the cards on hand are sorted. So, in insertion sort, each element is scanned and it is inserted in the proper position of previously sorted sub-array such that it is in sorted manner. To start with, we will assume that the array contains only one element and it is sorted trivially. Then we scan the second element of the array and it is compared with the first element and based on its value the second element will be kept at the left or right of first element such that two elements are sorted. This procedure is applied to rest of the elements of the array.

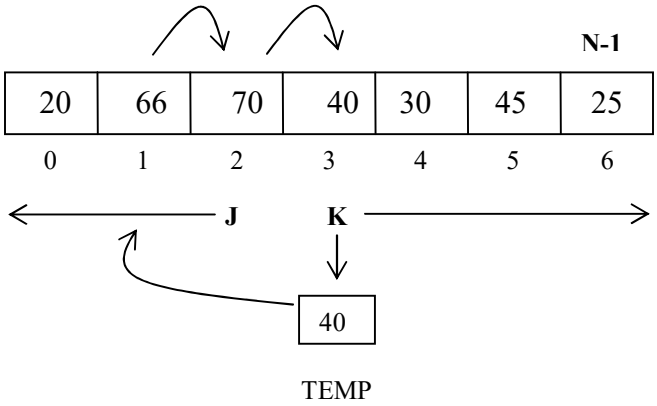
PHASE – I



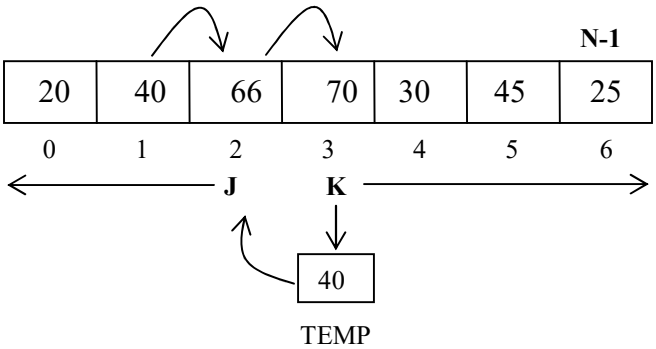
PHASE - II



PHASE - III



PHASE - IV



INSERTION_SORT (A [N])

```
Step 1: Let J, K, Temp
Step 2:     Set K: =1
Step 3:     Repeat for K: =1 to N-1 increasing by 1
            Set Temp: =A [K]
            J: =K-1
    Step3.1: Repeat While (J>=0) AND (Temp < A [J])
                A [J+1]:=A [J]
                J: =J-1
            [End of While]
    Step3.2: A [J+1]: = Temp
    [End of For]
Step 4:     Exit
```

6.2 SEARCHING

Searching is nothing but find the position of an element in the given list of elements. Searching techniques are of two types. They are:

1. Linear Search
2. Binary Search

6.2.1 Linear Search:

When an element is searched in sequence (i.e. from the beginning of the list to till the end of list), it is referred as linear search. Linear search is a time-consuming search especially when the element is present at the end of the list. The search can be either successful (when the element is found at some position) or unsuccessful (if the element is not found in the list) based on the presence or absence of the element in the given list.

Algorithm for Linear Search:
SEARCH (LA, LB, UB, ITEM)

[Assume LA is an array with list of elements, LB and UB represents the lower and upper bounds of the array LA. ITEM is the element to be searched in the list]

Step 1: Let K, FLAG
Step 2: Set K: =LB, FLAG: =0
Step 3: Repeat While (K<=UB)
    Step 3.1: If LA [K] =ITEM, Then
        Step 3.1.1: Set FLAG: =1
        Step 3.1.2: Display "Item Found at", K
        [End of IF]
    Step 3.2: Set K: =K+1
    [End of While]
Step 4: If FLAG=0, Then
    Step 4.1: Display "Item is not found"
    [End of IF]
Step 5: Exit

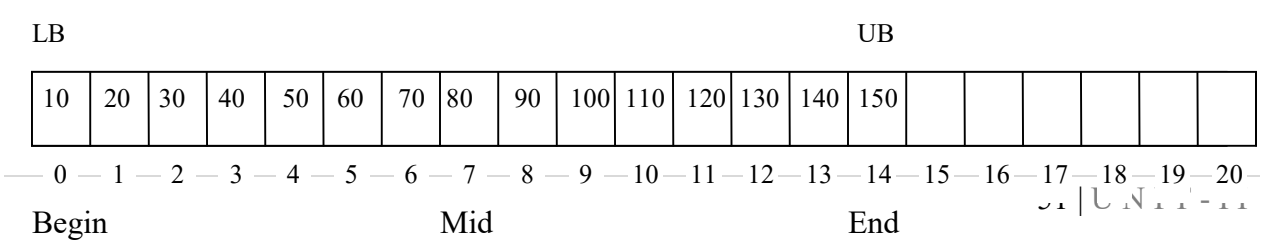
6.2.2 Binary Search

The binary search technique is useful as compared to the linear search but binary search requires the elements in a sorted order. In case of binary search the number of comparisons required to search an element is less as compared to linear search.

Binary search divides the list into two segments by selecting the middle position of the list. As the list is in sorted order, the element to be searched may present either left or right segment of the list based on its value (i.e. if the element is less than middle position value, then the element will be on left segment, otherwise it may present in right segment). This left or right segment is further divided into two segments and this process continues until the element is found or when only one left in the list.

Algorithm for Binary Search:

Let LA is a Linear array with size 20
Search for ITEM = 110



BINARY SEARCH (LA, UB, ITEM)

```
        LA      Sorted linear array
        LB      Lower bound
        UB      Upper bound
        ITEM        Element to be searched
        LOC     Location of ITEM
        BEG     Beginning Location of a Segment
        END     End location of a segment
        MID     Middle location of a segment

Step 1:     Let BEG, END, MID, LOC
Step 2:     Set BEG: =LB, END: =UB, MID :=( BEG+END)/2
Step 3:     Repeat While BEG<=END AND LA [MID]! =ITEM
    Step 3.1: If ITEM<LA [MID], Then
                Set END: =MID-1
    Step 3.2: Else
                Set BEG: =MID+1
            [End of if]
    Step 3.3: Set MID :=( BEG+END)/2
    [End of While – Step3]

Step 4:     If LA [MID] =ITEM, Then
            Set LOC: =MID
    Else
            Set LOC: =0

    [End of IF]
Step 5:     Exit
```

COMPLEXITY

7.1 COMPLEXITY

It is a scheme that allows to measure efficiency of an algorithm in general fashion. During complexity calculation it may not apply exactly to the real life situation. To measure the efficiency of an algorithm/program, there are two major factors exist:

1. Space Complexity
2. Time Complexity

7.1.1 Space Complexity

It is the amount of space required to run a program. The space needed by a program is

1. Space for the code
2. Space for Simple Variables
3. Space for fixed size variables
4. Space for Memory Used/allocated during execution.

7.1.2 Time Complexity

It is the amount of computer time needed to run a program.

It depends on the sum of compile time and runtime. So the time is depending on the machine where we are executing the program. Also we can not find exact time in each machine.

As it is difficult to get reliable time using clock, so we will concentrate on frequency count for all statements. So, during the calculation the time measure is counted by the no. of key operations, instructions.

Example1:
Let x = 0;
x= x+1;

Example2:
Let x= 0, n=10,i;
for(i=0;i<n;i++)

```
        x=x+1;
```

Example3:

```
Let x= 0, n=10,i,j;
for(i=0;i<n;i++)
        for(j=0;j<n;j++)
                x=x+1;
```

For the above examples we can find frequency count as,
Example1: Frequency Count is 1 i.e. x=1
Example2: Frequency Count is n i.e. x=10
Example3: Frequency Count is n^2 i.e. x=100

e.g. Let an Array A[10] with elements exist as:

7	9	45	20	20	49	29	27	5	11
0	1	2	3	4	5	6	7	8	9

While searching for an ITEM, it is possible that the ITEM may found at the beginning, at the end or in between at a specific location. Accordingly for any program, the execution is also based on the input values.

The possible cases to measure the frequency count of a program are:
Best Case : The situation where frequency count of the program for execution is minimum.
Worst Case : The situation where frequency count of the program for execution is maximum.
Average Case : (Best Case + Worst Case) / 2

For the above array A[10], to search for an ITEM :
In best case the no. of comparisons are : 1
In worst case the no. of comparisons are : 10
In average case it is : (1+10)/2 = 5

The complexity of an algorithm M with size of input data n is the function f(n).

So, best case is the minimum no. of possible values of f(n) for any possible input. Similarly the worst case is the maximum no. of possible values of f(n).

Let the function f(n) = g(n) in worst case. So, if the item is found before reaching to n,
then f(n) <=g(n)

The worst case time complexity is usually calculated by comparing some standard functions. Some common asymptotic functions are:
Constant : 1
Logarithmic : log n
Linear : n
Quadratic : n^2
Exponential : 2^n
Factorial : n!
Cubic : n^3

Rate of Growth Table:

n / g(n)	**Log n**	**N**	**n log n**	**n^2**	**n^3**	**2^n**
5	3	5	15	25	125	32
10	4	10	40	100	10^3	10^3
100	7	100	700	10^4	10^6	10^{30}
1000	10	10^3	10^4	10^6	10^9	10^{300}

7.2 ASYMPTOTIC NOTATIONS:

These notations we use to describe about the time complexity of an algorithm. The algorithms are defined in terms of functions whose domains are the set of natural numbers.
And to examine the growth rate of f(n) the asymptotic notations available are:

1. Big Omega (Ω) for best case
2. Big Theta (θ) for average case
3. Big Oh (O) used for worst case

7.2.3 O (Big Oh) Notation:

Consider a function f(n) which is non-negative for all integers n = 0. We say that f(n) is Big Oh g(n), which we write f(n) = O(g(n)), if there exists an integer n_0 and a constant c>0 such that for all integers n = n_0, f(n) = cg(n). It can be written as :

$O(g(n)) = \{ f(n)$: there exist positive constants c and n_o such that $0 <= f(n) <= cg(n)$ for all $n >= n_0 \}$

Worst case time complexity of some algorithms in data structures

Linear Search	:	$O(n)$
Binary Search	:	$O(\text{Log } n)$ or $O(\text{Log}_2 n)$
Bubble Sort	:	$O(n^2)$
Merge Sort	:	$O(n \log n)$
Quick Sort	:	$O(n \log n)$
Heap Sort	:	$O(n \log n)$
Insertion Sort	:	$O(n^2)$
Selection Sort	:	$O(n^2)$
Radix Sort	:	$O(n^2)$

Time complexity of the bubble Sort:

The first pass makes n-1 comparisons; the 2nd pass makes n-2 comparisons and so on. So the total complexity is:

$$
\begin{aligned}
f(n) &= (n-1)+(n+2)+\ldots\ldots+2+1 \\
&= (n(n-1))/2 \\
&= (n^2/2) - (n/2) \\
&= O(n^2)
\end{aligned}
$$

So, f(n) is purely based on the input value of n

So $f(n)=O(g(n))$

Based on Big Oh Notation, the algorithm can be categorized:

- Constant Time (O(i)) algorithm
- Logarithmic Time (O(logn)) algorithms.
- LinearTime (O(n)) algorithm
- Polynomial Time ($O(n^k)$, for K>1) algorithm
- Exponentail Time ($O(K^n)$, for K>1) algorithm

<u>Complexity Specified for Algorithm</u>

Linear Search:	$O(n)$
Binary Search:	$O(\text{Log } n)$ (Or) $O(\text{Log}_2 n)$
Bubble Sort :	$O(n^2)$
Merge Sort :	$O(n \log n)$

Quick Sort : $O(n \log n)$
Heap Sort : $O(n \log n)$
Insertion Sort: $O(n^2)$
Selection Sort: $O(n^2)$
Radix Sort : $O(n^2)$

Complexity (Linear Search)

Best Case : f(n) – small O(1)
Worst Case : f(n) – bigO(n)
Average Case: $\frac{\text{(Best Case + Worst Case)}}{2}$

i.e. $\frac{1+n}{2}$

Complexity of the Bubble Sort

$$F(n) = (n-1)+(n+2)+\ldots\ldots+2+1$$
$$= \frac{n(n-1)}{2} \quad \frac{n^2}{2} \quad \frac{n}{2} = O(n^2)$$

Selection Sort

The first pass makes n-1 comparisons, the 2nd pass makes n-2 comparisons and so on. So the total complexity is

$(n-1) +(n-2)+(n-3)+\ldots\ldots..+1$
$= n(n-1)/2$ Comparisons
so, $f(n) = O(n^2)$

SINGLY LINKED LIST

8.1 Introduction to Linked List

Linked List is a linear collection of data elements called nodes.

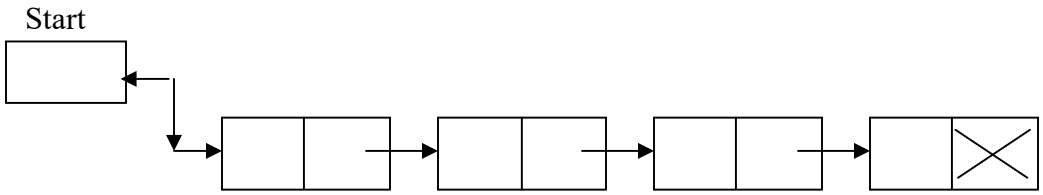
Node: A Node is a memory location that contains two parts.

1. Information of element(or) Data
2. Link Field contains the address of next Node.

Diagrammatically structure of a node look as follow:

INFO	LINK

(Structure of a node in linked list)



- START is an external pointer which identifies address of 1st Node i.e. the beginning of the linked list.
- The address of Link part of last node contains NULL as no further nodes exist.

8.2 DYNAMIC STORAGE MANAGEMENT

8.2.1 Static memory allocation:

Static memory allocation is the one in which the required memory is allocated at compile time. e.g: arrays are the best example of static memory allocation.

Drawbacks of Static Memory Allocation:

Static memory allocation has the following drawbacks.

1. Size is fixed before execution
2. Insertion and deletion is a time taking process as the elements need to be shifted towards left or right
3. There may be a possibility that memory may get wasted or memory is deficit.

8.2.2 Dynamic memory allocation:

Dynamic memory allocation is the one in which the required memory is allocated at run time. e.g: linked lists are the best example of dynamic memory allocation.

The drawbacks of static memory allocation can be overcome by dynamic memory allocation.

Advantages of Linked List:

A Linked list has the following advantages over arrays.

1. The allocated size can be increased or decreased as per the requirement at run time.
2. Memory can be utilized efficiently as the programmer can choose exact amount of memory needed.
3. The operations like insertion and deletion are less time-consuming as compared to arrays.

Disadvantages of Linked List:

A linked list has few disadvantages. They are:

1. It takes more memory space because each node contains the link or address part.
2. Moving to a specific node directly like array is not possible.

How to declare structure of a node in 'C' Language:

```
struct  node
{
        int info;
        struct node * link; // self referencing structure
};
struct node *p;
p = (struct node *) malloc (sizeof (struct node));
```

NOTE: If a pointer to a structure is declared as a structure element of the same structure, it is referred as "SELF REFERENCING STRUCTURE".

8.3 BASIC OPERATIONS ON LINKED LIST:

The basic operations that can be performed on a linked list are as follows:

1. Creation
2. Insertion
3. Deletion
4. Traversing
5. Searching
6. Sorting
7. Concatenation

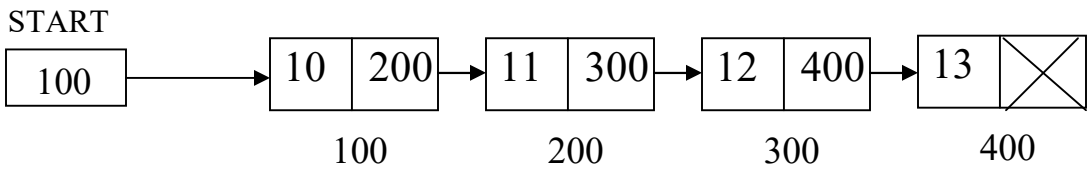
8.4 TYPES OF LINKED LIST:

Linked lists are of the following categories:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List
4. Circular Doubly Linked List

8.5 SINGLY LINKED LIST

A single linked list is a collection of nodes where each node contains the address of next node. In other words, it is also referred as *one-way* list as traversing is possible in only one direction i.e. from START to NULL



1) A Linear List having link part in each node.
2) Traverse can be from START to NULL and Backward traversing Not possible (i.e. one way List)

It is a linear collection of data item, where each element in the list contains a field called link, which contains the address of next element in the list.)

8.5.1 MEMORY ALLOCATION

1. Assume a set of free memory blocks in the form of nodes exist in the memory of the computer.
2. Assume AVAIL is a pointer which holds the address first free node in the memory.
3. Assume each node has INFO and LINK parts.

Suppose START, PTR and FRESH are the pointers, which are related to a single linked list.

1. START identifies the address of first node of linked list.
2. FRESH identifies the new node to be added to a linked list.
3. PTR will be used to move between linked lists.

When AVAIL=NULL

The condition AVAIL=NULL indicates that no free memory blocks (or nodes) exist in the memory. At this point of time if we try to create a new node overflow situation occurs.

When START=NULL

The condition START=NULL indicates that linked list has no nodes or linked list does not exist. At this point of time if we try to delete a node from linked list it leads to underflow situation.

The following pseudo code checks for availability of memory and a new node is prepared if free memory is available otherwise displays an overflow message.

Step 1: If AVAIL=NULL, Then

Display "Insufficient Memory or Overflow"

Exit

Step 2: Else

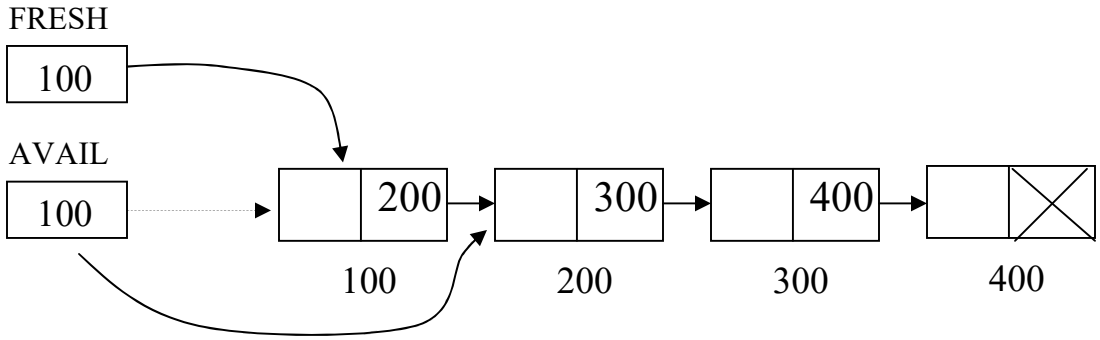
FRESH: =AVAIL [FRESH is assigned with the node as pointed by AVAIL]

AVAIL: =LINK [AVAIL] [AVAIL now holds address of next available free node]

INFO [FRESH]:=ITEM [INFO part of FRESH is updated with ITEM]

LINK [FRESH]:=NULL [Address or LINK part is assigned with NULL]

[End of IF]



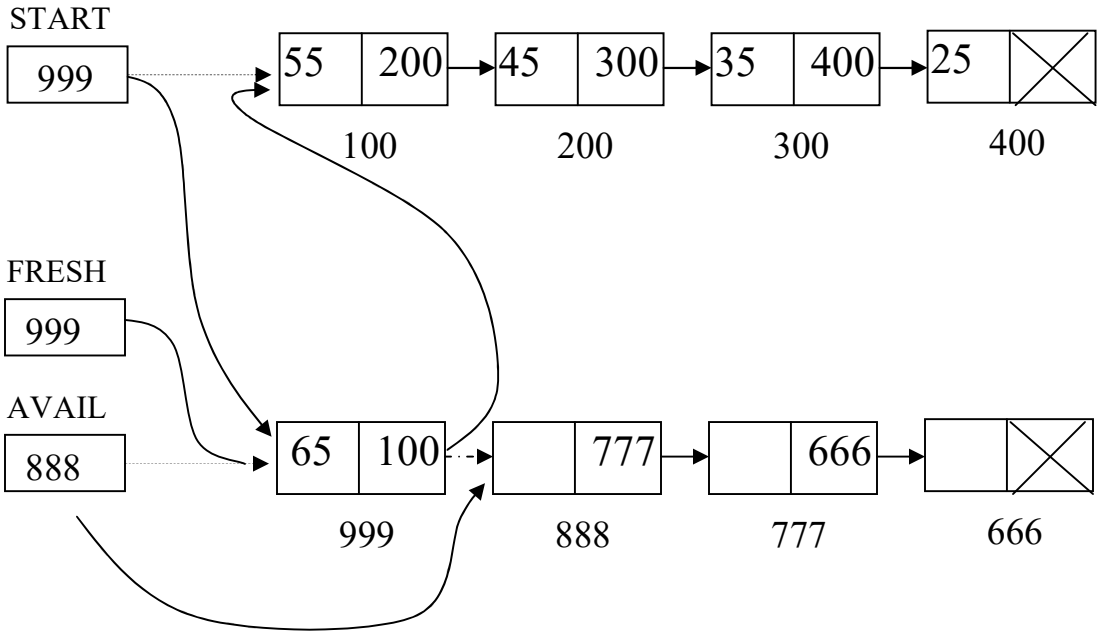
8.5.2 Operations on a single linked list:

The following operations are possible on a single linked list.

2) ***Insertion of a node in a single linked list***
   a) Insertion at the beginning
   b) Insertion at the end
   c) Insertion after specified node
3) ***Deletion of a node in a single linked list***
   a) Deletion at the beginning
   b) Deletion at the end
   c) Deletion of a node based on a given item (or INFO)
4) ***Searching for a node based on given item***
5) ***Concatenation of two linked lists***
6) ***Reversing the single linked list.***

8.5.3 INSERTION OPERATION ON SINGLE LINKED LIST:

8.5.3.1 Insertion at the beginning *(when linked list has one or more nodes)*

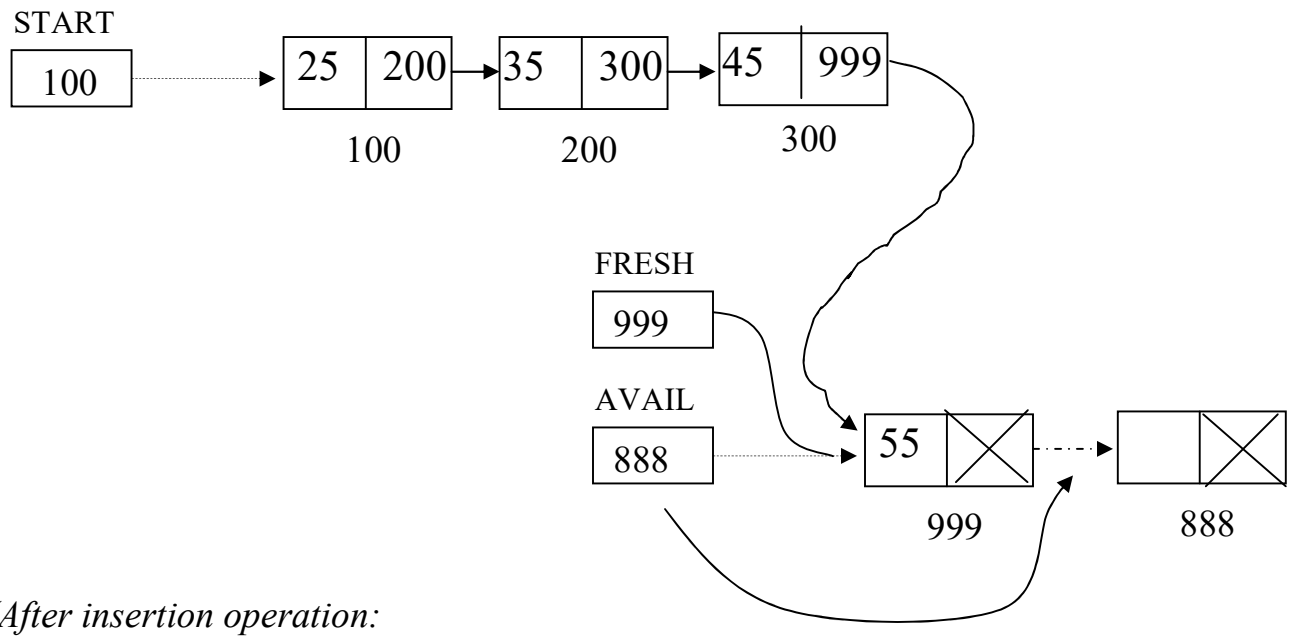


(START initially contains the 1st node address i.e.100,
After insertion operation:
START points to node with address 999
Node with address 999 points to the address 100
and AVAIL points to the next free node with address 888)

Algorithm:

```
INSERTFIRST (INFO, LINK, START, AVAIL, ITEM)
Step 1: If AVAIL=NULL, Then
            Display "Memory insufficient or Over flow"
            Exit.
        [End of IF]
Step 2: If START=NULL, Then
            START=FRESH
Step 3: Else
            FRESH: =AVAIL
            AVAIL: =LINK [AVAIL]
            INFO [FRESH]:=ITEM
            LINK [FRESH]:=START
            START: =FRESH
        [End of IF]
Step 3: Exit.
```

8.5.3.2 Insertion at the end of a Single Linked List



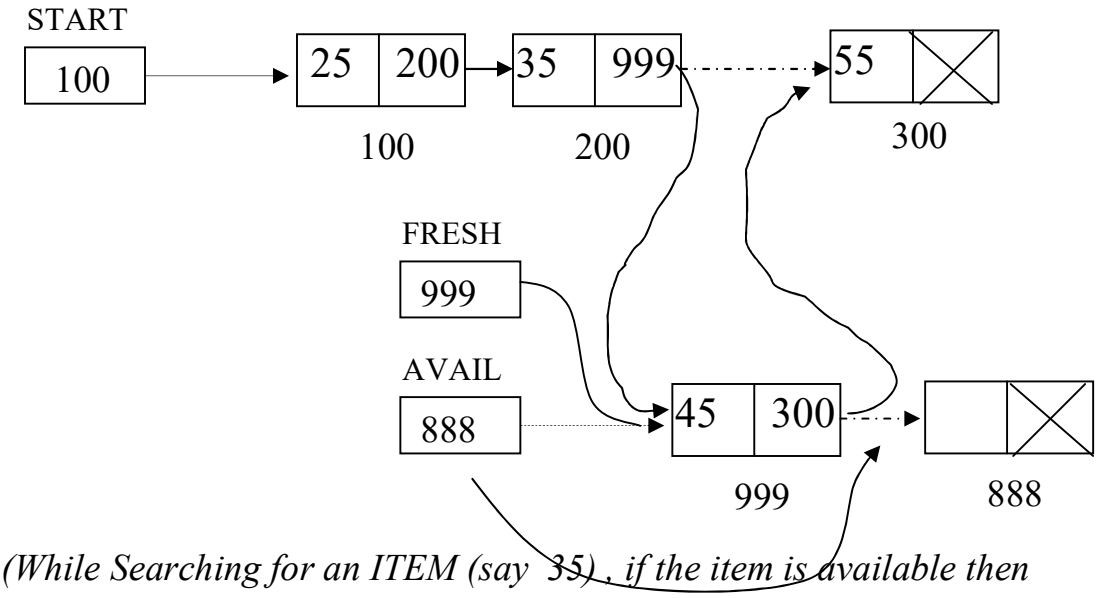
(After insertion operation:
The last node with address 300 points to node with address 999
And link part of Node with address 999 sets to NULL
and AVAIL points to the next free node with address 888)

Algorithm:

INSERTEND (INFO, LINK, START, AVAIL, ITEM)

Step 1: Let PTR
Step 2: If AVAIL=NULL, Then
Display "Memory Insufficient or Overflow"
Exit
Step 3: Else
FRESH: =AVAIL
AVAIL: =LINK [AVAIL]
INFO [FRESH]:=ITEM
LINK [FRESH]:=NULL
Step 3.1: If START=NULL, Then
START: =FRESH
Step 3.2: Else
PTR: =START
Repeat While LINK [PTR] ! = NULL [To make PTR point Last node]
PTR: =LINK [PTR]
[End of While]
LINK [PTR]:=FRESH
[End of IF- Step 3.1]
[End of IF – Step 2]
Step 4: Exit

8.5.3.3 Insertion of a node AFTER specified Node in single linked list:



(While Searching for an ITEM (say 35) , if the item is available then

insertion after that node is accomplished as follow:
Here the node with address 200 contains ITEM (i.e.35). So node with address 200 points to new node with address 999 and the link part of node with address 999 points to the node with address 300.
So, now the new node is inserted after the node having info part with value 35. and AVAIL points to the next free node with address 888)

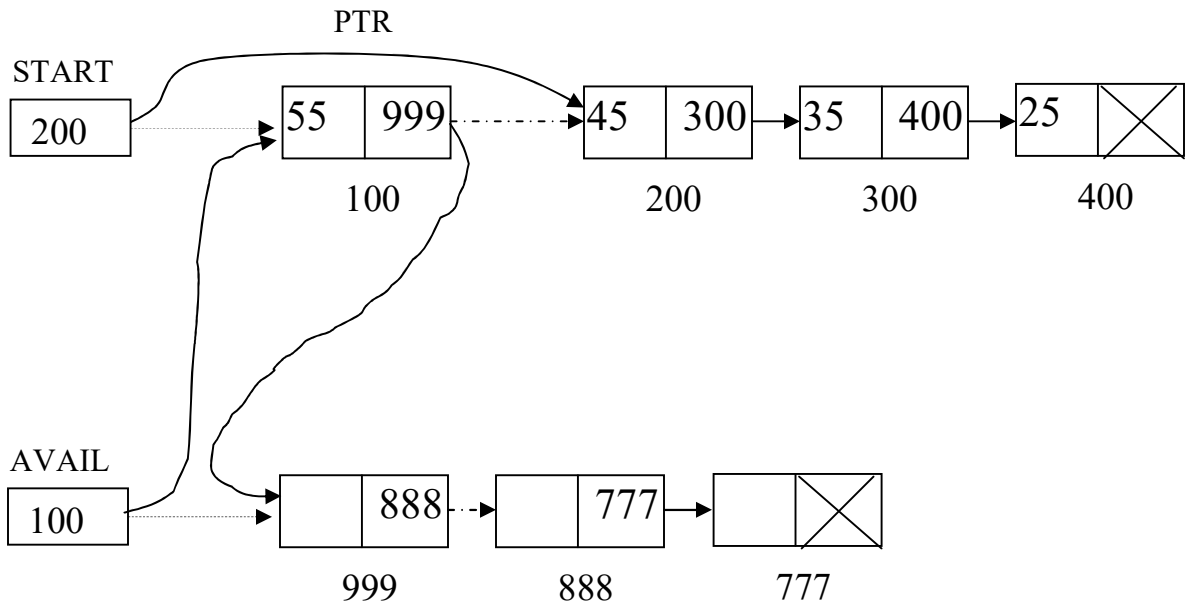
Algorithm:
INSERTAFTERNODE (INFO, LINK, START, ITEM, PTR, AVAIL, NEWITEM, FRESH)

```
Step 1: Let F: =0
Step 2: If AVAIL=NULL, then
        Display "Memory Insufficient or Overflow"
        Exit
Step 3: Else
        FRESH: =AVAIL
        AVAIL: =LINK [AVAIL]
        INFO [FRESH]:=NEWITEM
        LINK [FRESH]:=NULL
        Step 3.1: If START=NULL, then
                Display "List does not exist"
        Step 3.2: Else
                PTR: =START
                Repeat While PTR ! =NULL
                    If ITEM=INFO [PTR], then
                        F: =1
                        BREAK
                    [End of IF]
                    PTR: =LINK [PTR]
                [End of While]
                Step 3.2.1: If F=1, then
                            LINK [FRESH]:=LINK [PTR]
                            LINK [PTR]:= FRESH
                Step 3.2.2: Else
                            Display"Item not Found"
                        [End of IF]
        [End of IF]
Step 3: EXIT
```

8.5.4 DELETION FROM A LINKED LIST:

While performing deletion operation in a single linked list, the memory space of the node which will be deleted, should be de-allocated so that the operating system collects free space and this memory space is added to the AVAIL list. Specifically for easier processing it will be added at the beginning the AVAIL list.

8.5.4.1 Deletion of first node from a single linked list

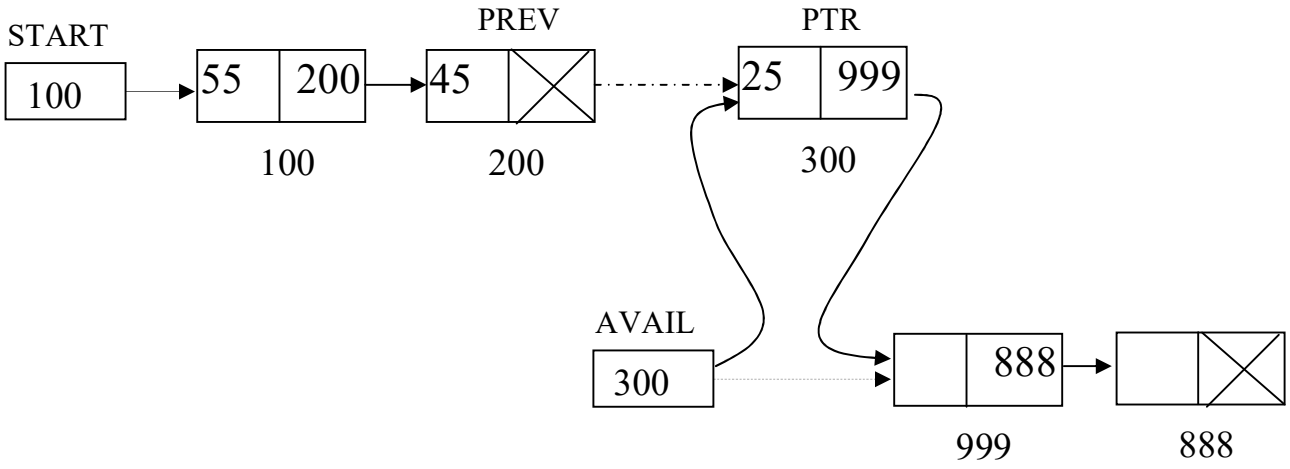


Algorithm:

DELETION FIRST (INFO, LINK, START, AVAIL, PTR)

Step 1: If START=NULL, Then
Display "List does not exist OR Underflow"
Step 2: Else
PTR: =START
START: =LINK [PTR]
LINK [PTR]:=AVAIL
AVAIL: =PTR
[End of IF]
Step 3: EXIT

8.5.4.2 Deletion of last node from single linked list



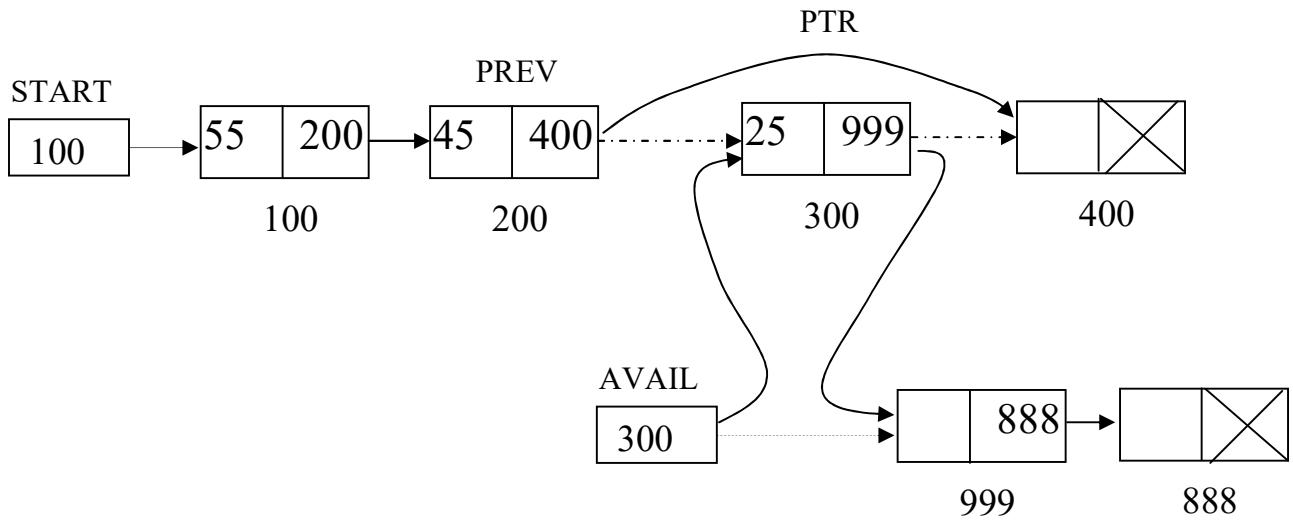
Algorithm:

DELETION END (START, LINK, INFO, PTR, PREV, AVAIL)

Step 1: PTR: =START
Step 2: Repeat While LINK [PTR] != NULL
PREV: =PTR
PTR: =LINK [PTR]
[End of While]

Step 3: LINK [PREV]:=NULL
Step 4: LINK [PTR]:=AVAIL
Step 5: AVAIL: =PTR
Step 6: EXIT

1.5.4.3 Deletion of a node based on a given item

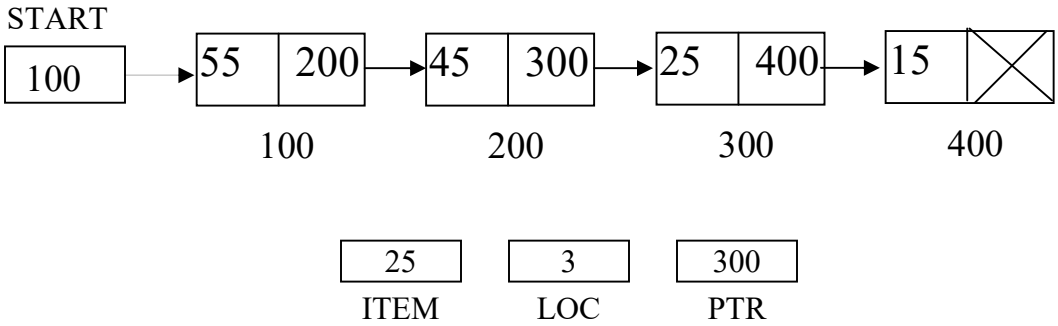


Algorithm:

DELETION ITEM (LINK, INFO, START, PTR, PREV, AVAIL, ITEM)

Step 1: PTR: =START
Step 2: Repeat While PTR! =NULL AND ITEM! =INFO [PTR]
    PREV: =PTR
    PTR: =LINK [PTR]
  [End of While]
Step 3: If PTR=NULL, Then
    Display "Item Not Found"
Step 4: Else If PTR=START, Then
    START: =LINK [PTR]
Step 5: Else
    LINK [PREV]:=LINK [PTR]
  [End of IF]
Step 6: LINK [PTR]:=AVAIL
Step 7: AVAIL: =PTR
Step 8: EXIT

8.5.5 SEARCH OF A NODE IN A SINGLE LINKED LIST (BASED ON GIVEN ITEM)



Algorithm:

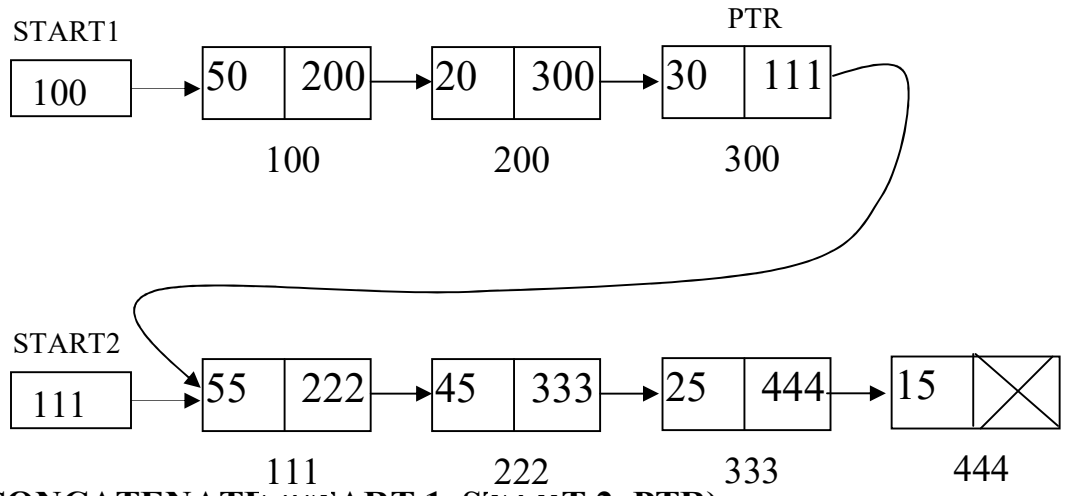
SEARCHNODE (INFO, LINK, START, PTR, ITEM)

```
Step 1: Let LOC: =0, F: =0
Step 2: PTR: =START
Step 3: Repeat While PTR! =NULL
            Step 3.1: Set LOC: =LOC + 1
            Step 3.2: If ITEM=INFO [PTR], then
                            F: =1
                            BREAK
                      [End of IF]
                      PTR: =LINK [PTR]
        [End of While]
Step 4: If F=0, Then
            Display "Item Not Found"
Step 5: Else
            Display "Item Found at Location", LOC
        [End of IF]
Step 6: EXIT
```

8.5.6 CONCATENATION OF TWO SINGLE LINKED LISTS

Algorithm:

[Assume two linked lists. START 1 identifies starting Node of 1st linked list and START 2 identifies starting Node of 2nd linked list. This algorithm joins 2nd linked list at the end of into 1st linked list]



CONCATENATE (START 1, START 2, PTR)

Step 1: PTR: =START 1

Step 2: Repeat While LINK [PTR]! =NULL

PTR: = LINK [PTR]

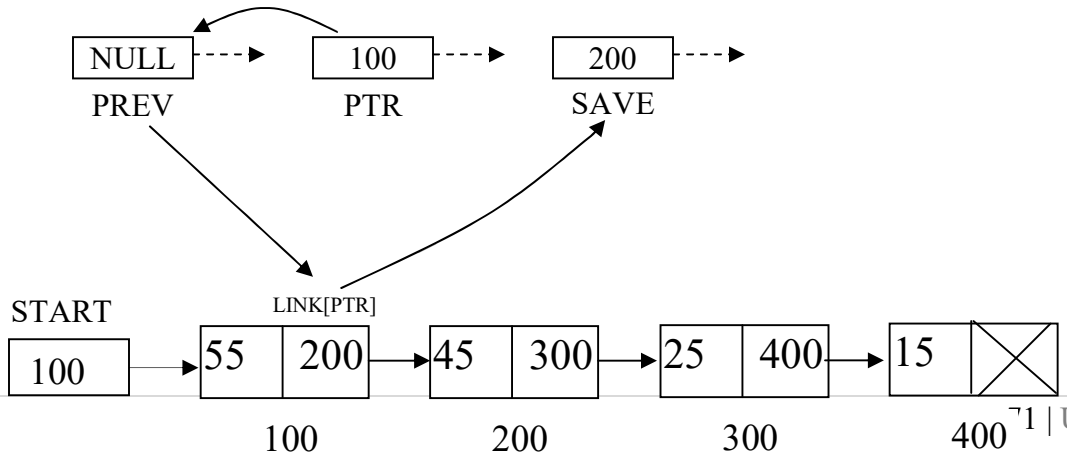
[End of While]

Step 3: LINK [PTR]:=START 2

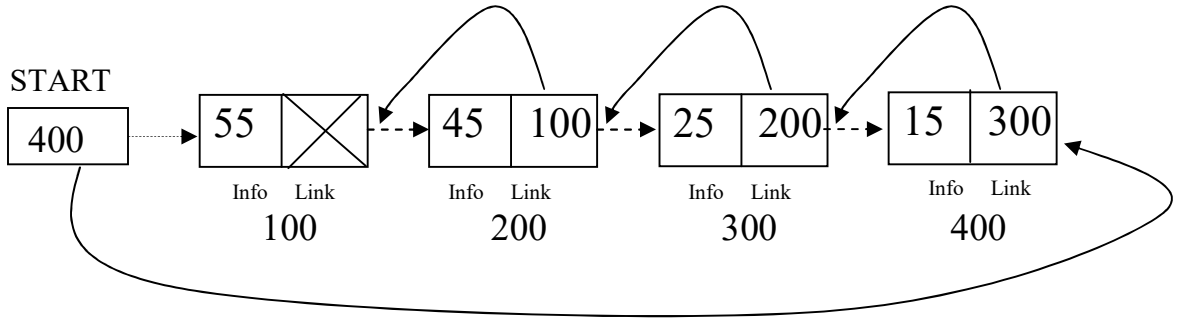
Step 4: EXIT

8.5.7 REVERSING A SINGLE LINKED LIST

Reversing a single linked list means pointing the START pointer to the last node, then the last node link part have to point to its previous node, and that node have to point to its previous node and so on till 1st node, where the 1st node link part will point to NULL.



Finally we get the linked list as :



Algorithm:

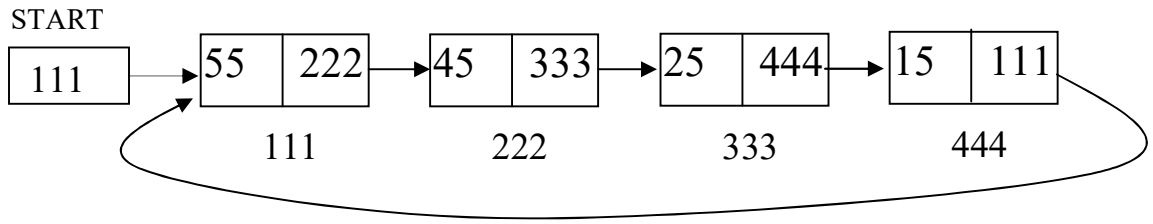
REVERSE (START, INFO, LINK)

Step 1: Let PREV, SAVE, PTR
Step 2: PTR: =START
Step 3: PREV: =NULL
Step 4: Repeat While PTR! =NULL
SAVE: =LINK [PTR]
LINK [PTR]:=PREV
PREV: =PTR
PTR: =SAVE
[End of While]
Step 5: START: =PREV
Step 6: Exit.

CIRCULAR SINGLY LINKED LIST

9.1 CIRCULAR SINGLY LINKED LIST

It is a single linked list where the link part of last node contains the address of first node. In other words when we traverse a circular single linked list the first node can be visited after the last node. As the circular single linked list of circular in nature as a result of which there will be no node whose address part i.e. link part with NULL.



9.2 Operations on Circular Single Linked List

<u>Insertion</u>

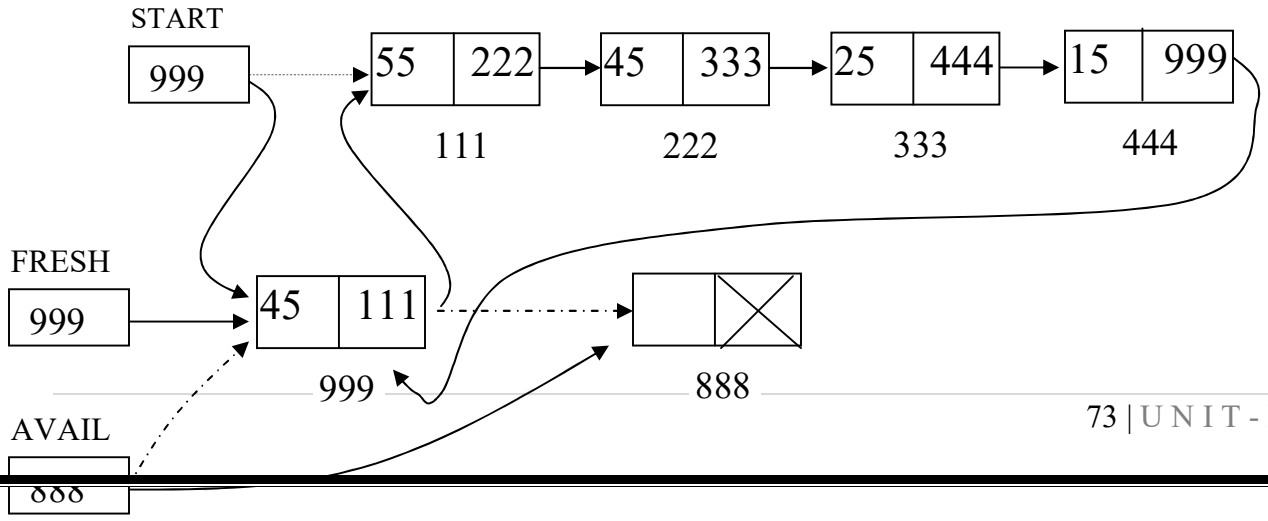
2. At the Beginning
3. At end.

<u>Deletion</u>

1. At the beginning
2. At the end.

9.2.1 INSERTION OPERATION IN A CIRCULAR SINGLE LINKED LIST

9.2.1.1 Insertion of a node at the beginning



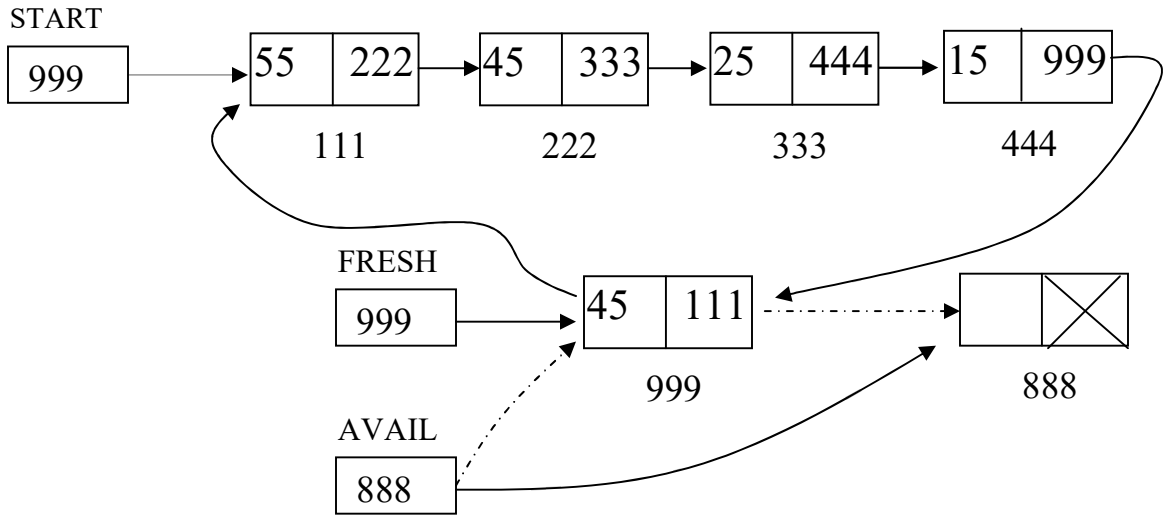
Algorithm:

INSERTFIRST (START, INFO, LINK, AVAIL, FRESH, ITEM)

```
Step 1: If AVAIL=NULL, Then
            Display "Memory Insufficient"
Step 2: Else
            FRESH: =AVAIL
            AVAIL: =LINK [AVAIL]
            INFO [FRESH]:=ITEM
            LINK [FRESH]:=NULL

            Step 2.1: If START=NULL, Then
                        START: =FRESH
                        LINK [FRESH]:=START
            Step 2.2: Else
                        LINK [FRESH]:=START
                        PTR: =START
                        Repeat While LINK [PTR] ! =START
                            PTR: =LINK [PTR]
                        [End of While]
                        LINK [PTR]:=FRESH
                        START: =FRESH
                    [End of IF]
Step 3: EXIT
```

9.2.1.2 Insertion of a node at the end



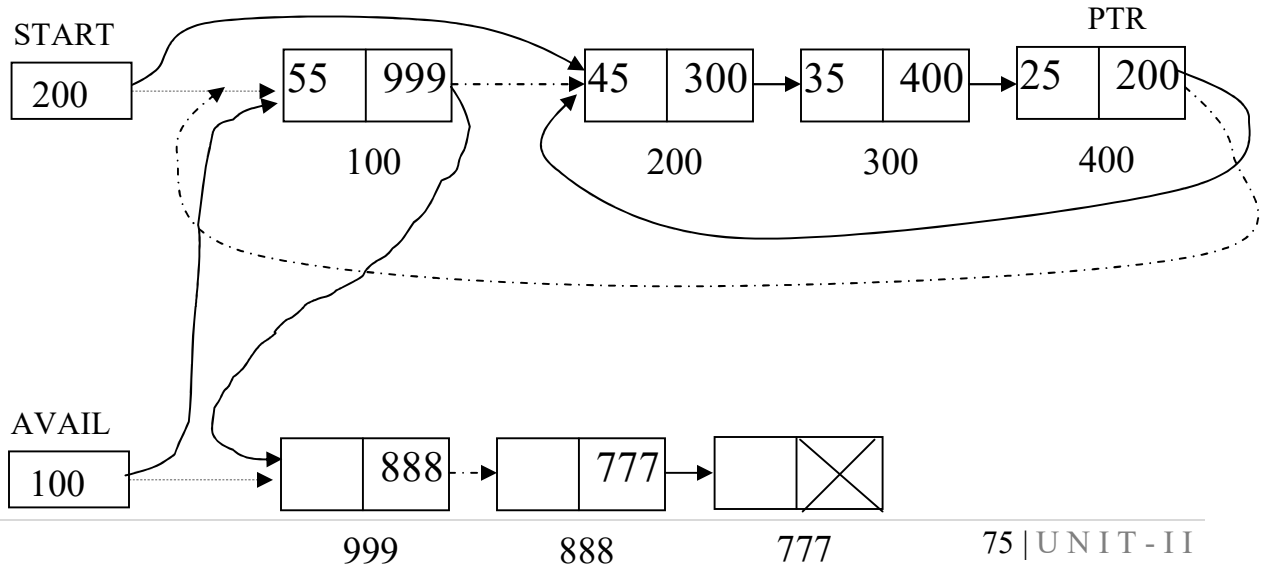
Algorithm:

INSERT END (START, INFO, LINK, AVAIL, FRESH, ITEM)

```
Step 1: If AVAIL=NULL, Then
            Display "Memory Insufficient"
Step 2: Else
            FRESH: =NULL
            AVAIL: =LINK [AVAIL]
            INFO [FRESH]:=ITEM
            LINK [FRESH]:=NULL
            Step 2.1: If START=NULL, then
                        START: =FRESH
                        LINK [FRESH]:=START
            Step 2.2: Else
                        PTR: =START
                        Repeat, While LINK [PTR]=START
                            PTR: =LINK [PTR]
                        [End of While]
                        LINK [PTR]:=FRESH
                        LINK [FRESH]:=START
                    [End of IF– Step 2.1]
        [End of IF – Step 1]
Step 5: Exit
```

9.2.2 DELETION OPERATION ON A CIRCULAR SINGLE LINKED LIST

9.2.2.1 Deletion of a node from the beginning

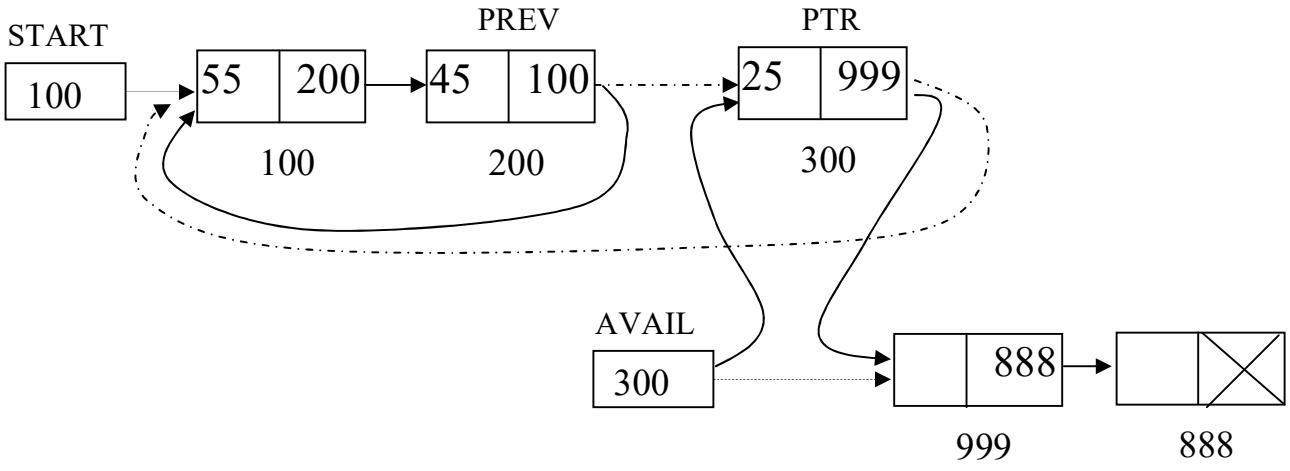


Algorithm:

DELETE FIRST (START, INFO, LINK, AVAIL)

```
Step 1: If START=NULL, Then
            Display "List does not exist"
Step 2: Else if          START=LINK [START], then
            LINK [START]:=AVAIL
            AVAIL: =START
            START: =NULL
Step 3: Else
            PTR: =START
            Repeat, While LINK [PTR]=START
                PTR: =LINK [PTR]
            [End of While]
            LINK [PTR]:= LINK [START]
            PTR=START
            START: =LINK [START]
            LINK [PTR]:=AVAIL
            AVAIL: =PTR
        [End of IF]
Step 4: EXIT
```

9.2.2.2 Deletion of a node from the end



Algorithm:

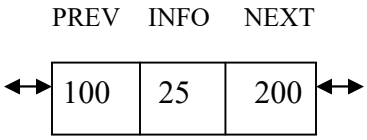
```
DELETE END (START, INFO, LINK, AVAIL)
Step 1: If START=NULL, Then
            Display "List Not Exist"
Step 2: Else If START=LINK [START], Then
            LINK [START]:=AVAIL
            AVAIL: =START
            START: =NULL
Step 3: Else
            PTR: =START
            Repeat, While LINK [PTR]=START
                PREV: =PTR
                PTR: =LINK [PTR]
            [End of While]
            LINK [PREV]:=START
            LINK [PTR]:=AVAIL
            AVAIL: =PTR
        [End of IF]
Step 4: EXIT.
```

DOUBLY LINKED LIST

10.1 DOUBLY LINKED LIST

A doubly linked list is a collection of nodes where each node contains the address of next as well as address of previous node along-with INFO part to hold the information. In other words, it is also referred as *two-way list* as traversing is possible in both the directions.

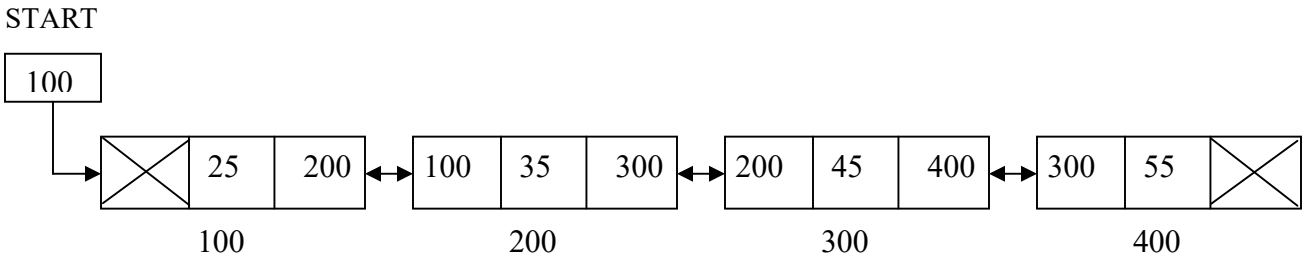
<u>Node of a Doubly Linked List</u>



Structure of a node in 'C'

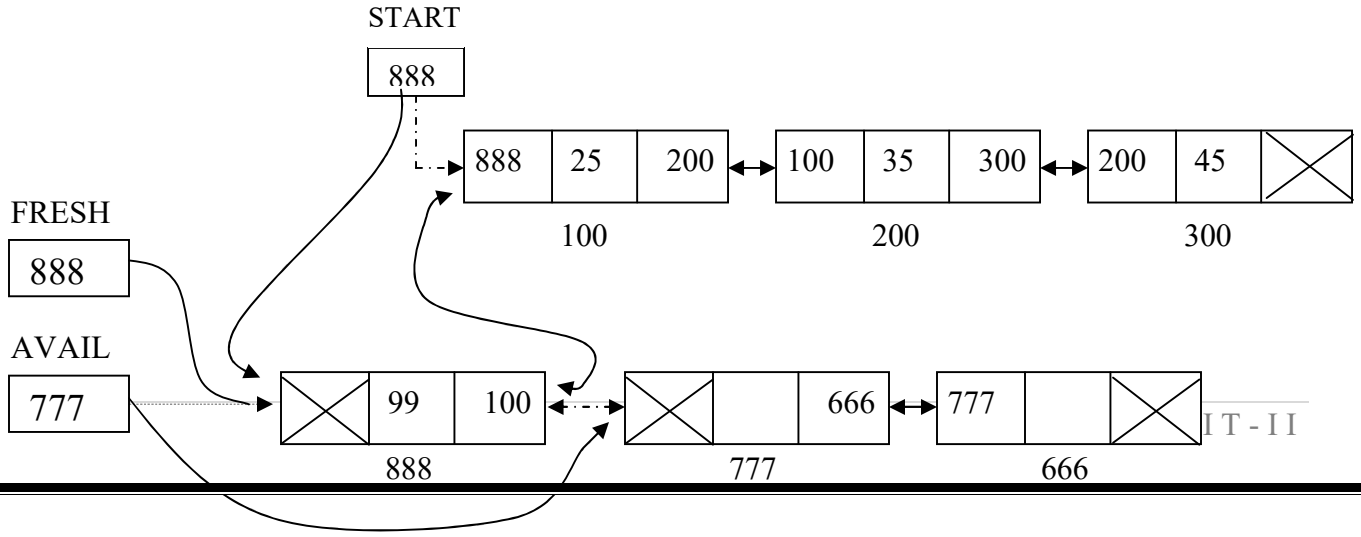
```
struct node
{       int info
        struct node *prev, *next;
};
```

A simple diagram of a doubly linked list as follows:



10.2 OPERATIONS ON DOUBLY LINKED LIST:

10.2.1 Insertion of a node at the beginning

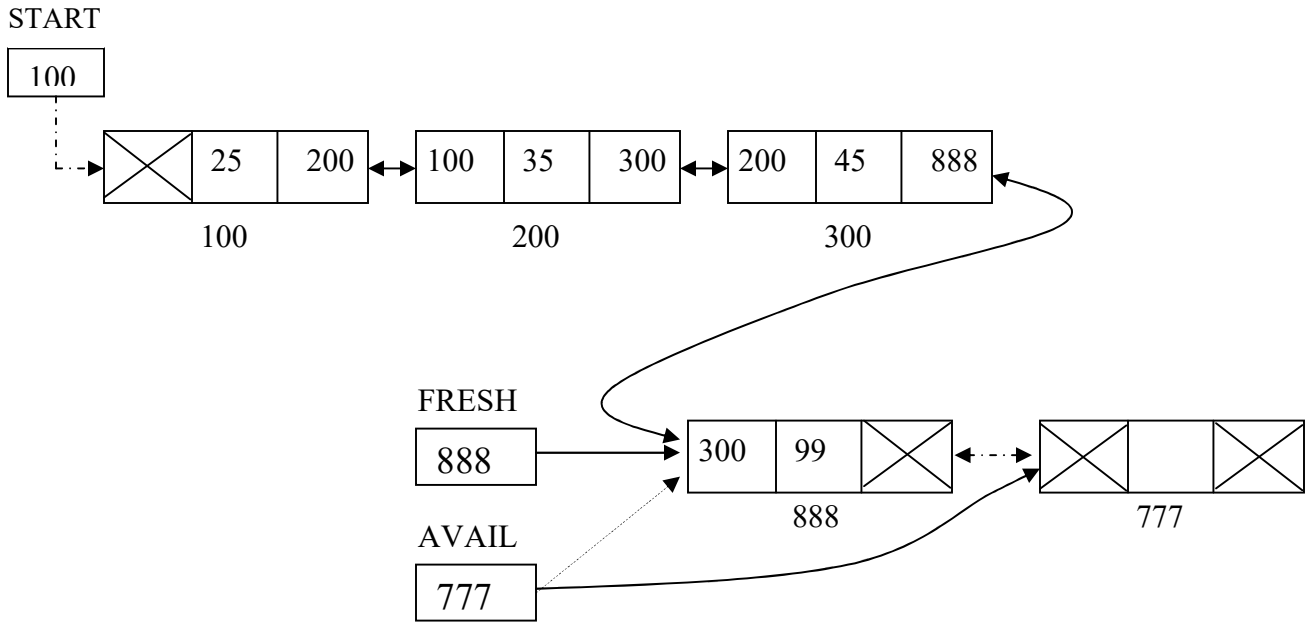


Algorithm:

INSERT FIRST (START, INFO, NEXT, PREV, FRESH, AVAIL)

```
Step 1: If AVAIL=NULL, Then
            Display "Insufficient Memory"
Step 2:         Else
            FRESH=AVAIL
            AVAIL: =NEXT [AVAIL]
            PREV [AVAIL]:=ITEM
            NEXT [FRESH]:= NULL
            PREV [FRESH]:= NULL
            INFO [FRESH]:=ITEM
        Step 2.1: If START=NULL, Then
                      START: =FRESH
        Step 2.2: Else
                      NEXT [FRESH]:=START
                      PREV [START]:=FRESH
                      START: =FRESH
                  [End of IF – Step 2.1]
        [End of if – Step 1]
Step 3: Exit
```

10.2.2 Insertion of a node at the end:



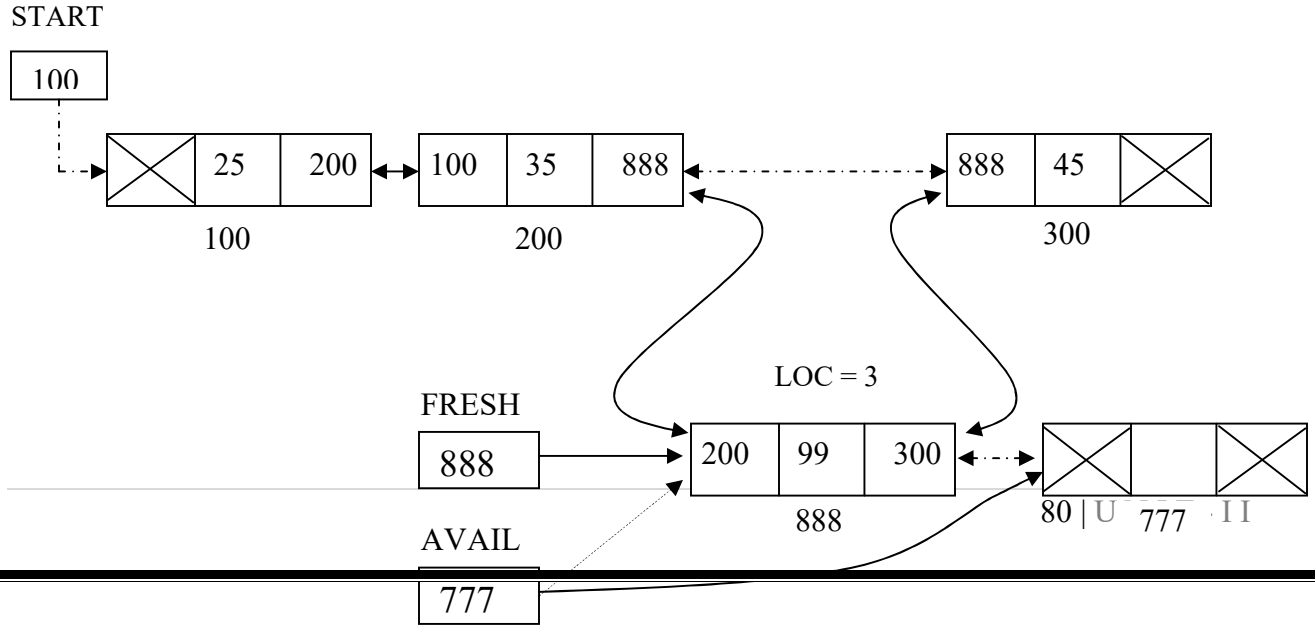
Algorithm:

```
INSERT END (START, INFO, NEXT, PREV, FRESH, AVAIL, PTR)
Step 1: If AVAIL=NULL, Then
            Display" Insufficient Memory"
Step 2:         Else
            FRESH=AVAIL
            AVAIL: =NEXT [AVAIL]
            PREV [AVAIL]:=ITEM
            NEXT [FRESH]:= NULL
            PREV [FRESH]:= NULL
            INFO [FRESH]:=ITEM
        Step 2.1: If START=NULL, Then
                        START: =FRESH

        Step 2.2: Else
                        PTR: =START
                        While NEXT [PTR]! =NULL
                                PTR: =NEXT [PTR]
                        [End of While]
                        NEXT [PTR]:=FRESH
                        PREV [FRESH]:=PTR
                    [End of IF – Step 2.1]
        [End of IF – Step 1]
Step 3: Exit
```

10.2.3 Insertion of a node at specific location

For Example: When Location LOC = 3, after insertion operation:



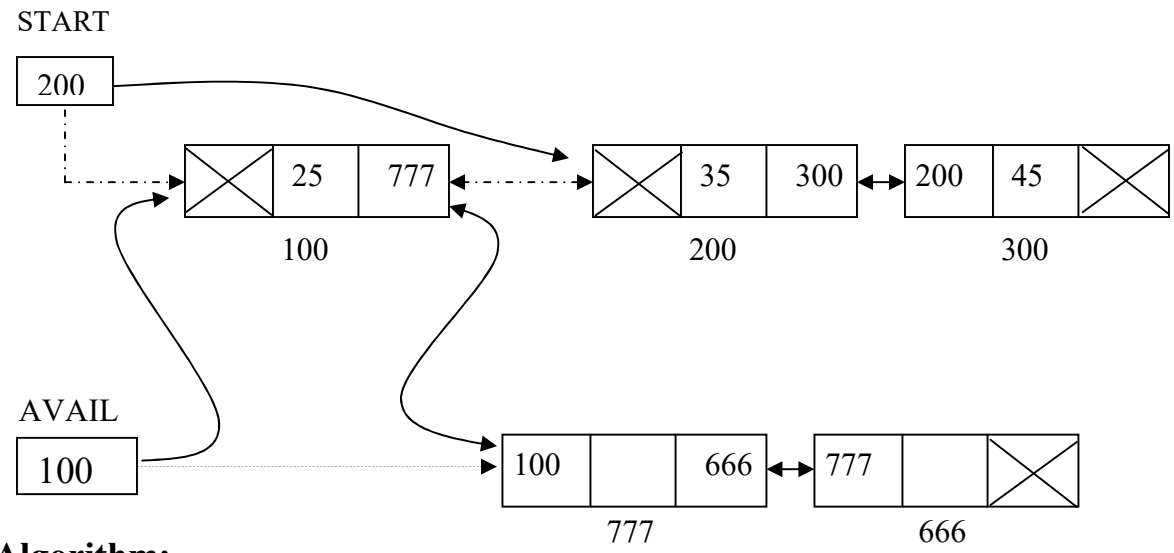
Algorithm:

INSERT_AT_LOC (START, INFO, NEXT, PREV, FRESH, AVAIL, PTR)

[In this algorithm, say PTR1 identifies the node at the specified position and PTR identifies previous Node]

```
Step 1: If AVAIL=NULL, Then
            Display" Insufficient Memory"
Step 2:         Else
            FRESH=AVAIL
            AVAIL: =NEXT [AVAIL]
            PREV [AVAIL]:=ITEM
            NEXT [FRESH]:= NULL
            PREV [FRESH]:= NULL
            INFO [FRESH]:=ITEM
        Step 2.1: If START=NULL, Then
                        START: =FRESH
        Step 2.2: Else
                        Step 2.2.1: Set I: =1, PTR1:=START
                        Step 2.2.2: Repeat While I<LOC AND PTR1! =NULL
                                        Set PTR: = PTR1
                                        Set PTR1:= NEXT [PTR1]
                                        Set I: =I + 1
                                    [End of While]
                        Step 2.2.3: If PTR1=NULL, Then
                                        Display "Location does not exist"
                        Step2.2.4: Else If PTR1=START, Then
                                        NEXT [FRESH]:=START
                                        PREV [START]:=FRESH
                                        START: =FRESH
                        Step 2.2.5: Else
                                        NEXT [PTR]:=FRESH
                                        PREV [PTR1]:=FRESH
                                        PREV [FRESH]:=PTR
                                        NEXT [FRESH]:=PTR
                                    [End of If – Step 2.2.3]
                [End of If – Step 2.1]
        [End of If – Step 1]
Step3.Exit
```

10.2.4 Deletion of a node from the beginning

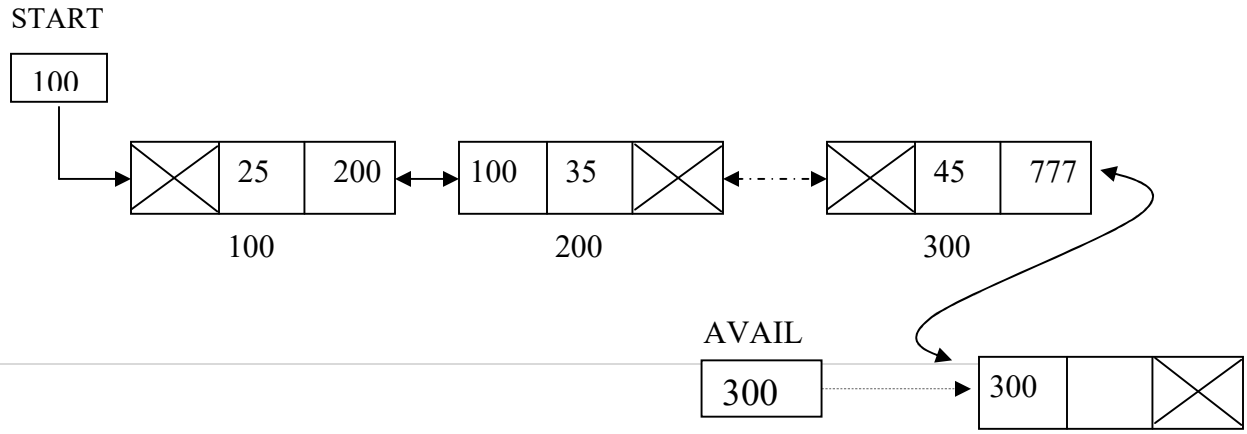


Algorithm:

DELETE FIRST (START, PREV, NEXT, PTR, AVAIL)

Step 1: If START=NULL, Then
Display "Linked List Does Not Exist OR Underflow"
Step 2: Else
PTR: =START
START: =NEXT [PTR]
NEXT [PTR]:=AVAIL
PREV [AVAIL]:=PTR
AVAIL: =PTR
[End of If]
Step 3: Exit

10.2.5 Deletion of a node from the end

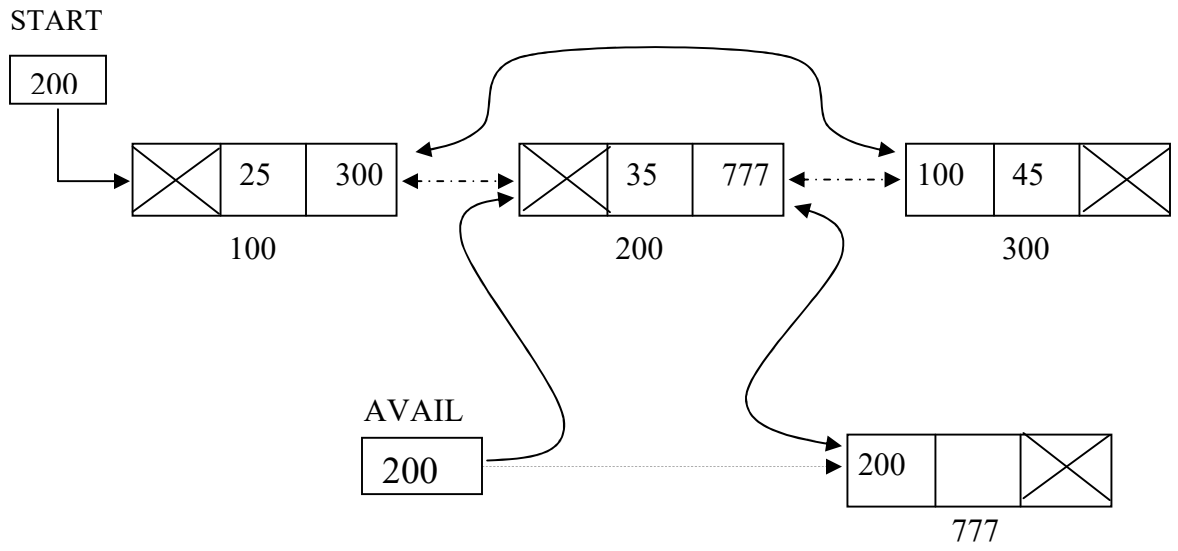


Algorithm:

DELETE END (START, PREV, NEXT, PTR, PTR1, AVAIL)

```
Step 1: If START=NULL, Then
            Display "Linked List Does Not Exist OR Underflow"
Step 2: Else
            PTR1:=START
            Step 2.1: Repeat While NEXT [PTR1]! =NULL
                        Set PTR: =PTR1
                        Set PTR1:=NEXT [PTR1]
                    [End of While]
            NEXT [PTR]:=NULL
            PREV [PTR1]:=NULL
            NEXT [PTR1]:=AVAIL
            PREV [AVAIL]:=PTR1
            AVAIL: =PTR1
        [End of If]
Step 3: Exit
```

10.2.6 Deletion of a node from specific position



Algorithm:

DELETEPOS (START, LOC, PTR, PTR1, ITEM, AVAIL)

```
Step 1: Let I: =1
Step 2: If START=NULL, Then
            Display'List not exist"
Step 3:         Else
            Step 3.1: PTR1:=START
            Step 3.2: Repeat While I<LOC AND PTR1! =NULL
                              PTR: =PTR1
                              PTR1:=NEXT [PTR1]
                              I: =I+1
                      [End of While]
            Step 3.3: If       PTR1:=START, Then
                              START: =NEXT [START]
                              PREV [START]:=NULL
            Step 3.4: Else If NEXT [PTR1] =NULL, Then
                              NEXT [PTR]:=NEXT [PTR1]
            Step 3.5: Else
                              PREV [NEXT [PTR1]]:=PTR
                              NEXT [PTR]:=NEXT [PTR1]
                      [End of If – Step 3.3]
            Step 3.6: NEXT [PTR1]:=AVAIL
                      PREV [PTR1]:=NULL
                      PREV [AVAIL]:=PTR1
                      AVAIL: =PTR1
        [End of If – Step 2]
Step 4:         Exit
```

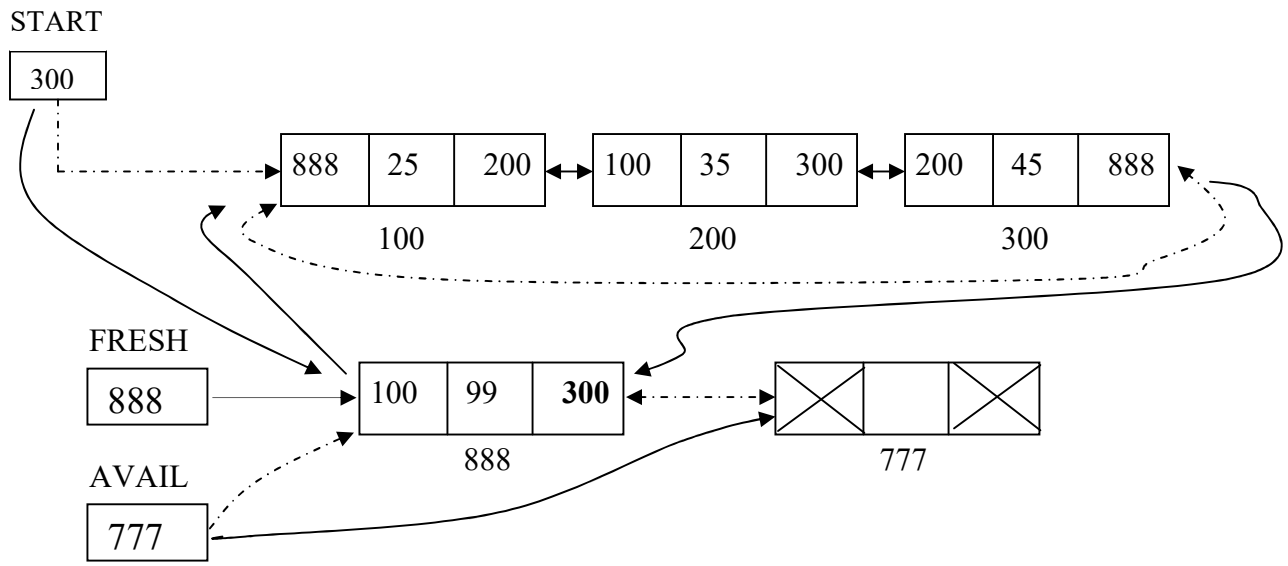
CIRCULAR DOUBLY LINKED LIST

11.1 Circular doubly linked list

The doubly linked list of circular in nature is referred as circular doubly linked list. i.e. In a circular doubly linked list, the NEXT part of last node contains the address of first node and the PREV part of first node contains address of last node.

11.2 Operations on circular doubly linked list:

11.2.1 Insertion of a node at the beginning



APPLICATIONS OF LINKED LIST

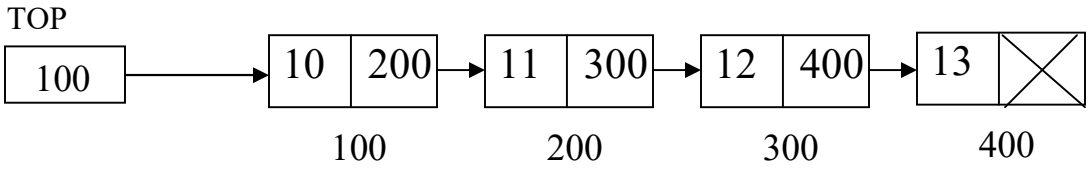
12.1 Applications of linked list:

A single linked list can be used to represent the following in the memory.

1. Linked Stack
2. Linked Queue
3. Polynomial Operations

12.2 Linked Stack

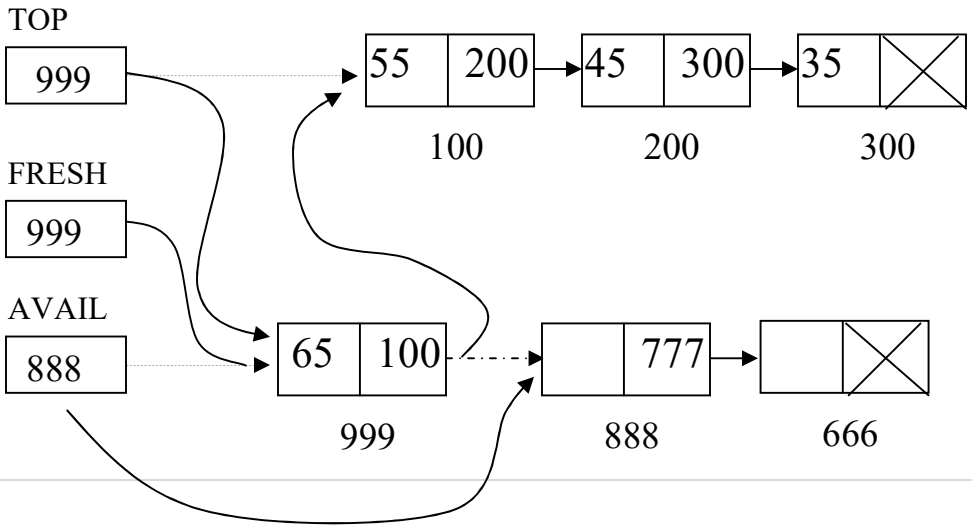
When the concept of stack is implemented using singly linked list, it is referred as linked stack. In linked stack, assume TOP is the pointer that identifies starting node. (Like START in single linked list).



PUSH Operation in a linked stack is similar to Insertion of a node at the beginning (i.e at Top of linked stack) of single linked list.

POP Operation in a linked stack is similar to Deletion of a node from the beginning (i.e. from Top of linked stack) of a single linked list.

12.2.1 PUSH OPERATION IN A LINKED STACK:

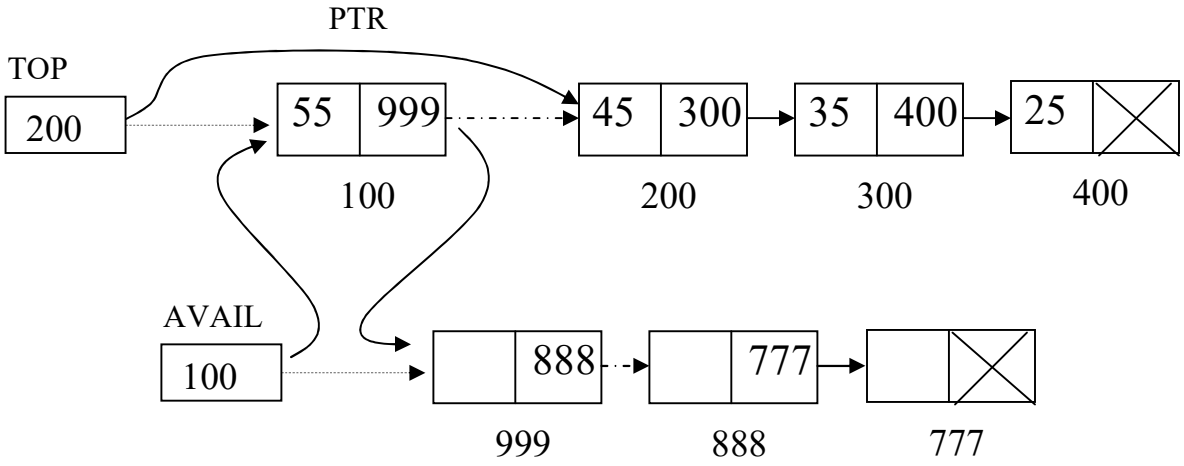


Algorithm:

PUSH (AVAIL, FRESH, LINK, INFO, TOP)

```
Step 1: If AVAIL=NULL, Then
            Display "Insufficient Memory OR Overflow"
Step 2: Else
            FRESH: =AVAIL
            AVAIL: =LINK [AVAIL]
            LINK [FRESH]:=NULL
            INFO [FRESH]:=ITEM
      Step 2.1: If TOP=NULL, Then
                  TOP: =FRESH
      Step 2.2: Else
                  LINK [FRESH]:=TOP
                  TOP: =FRESH
              [End of If – Step 2.1]
      [End of If – Step 1]
Step 3: Exit
```

12.2.2 Pop operation in a linked stack:

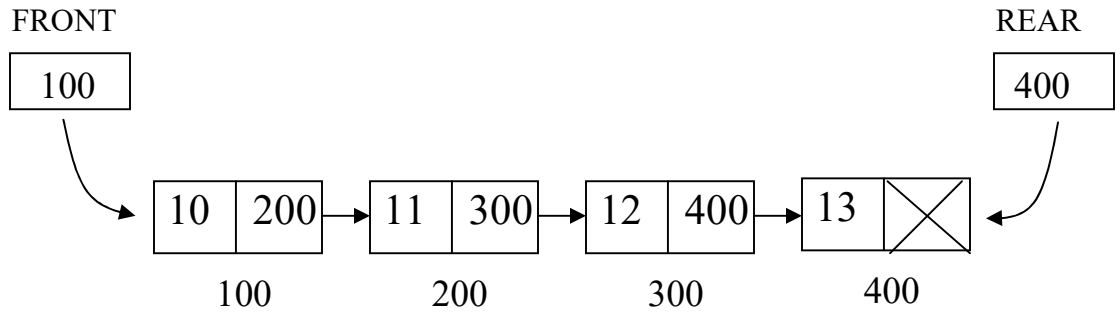


Algorithm:
POP (TOP, PTR, LINK, AVAIL)

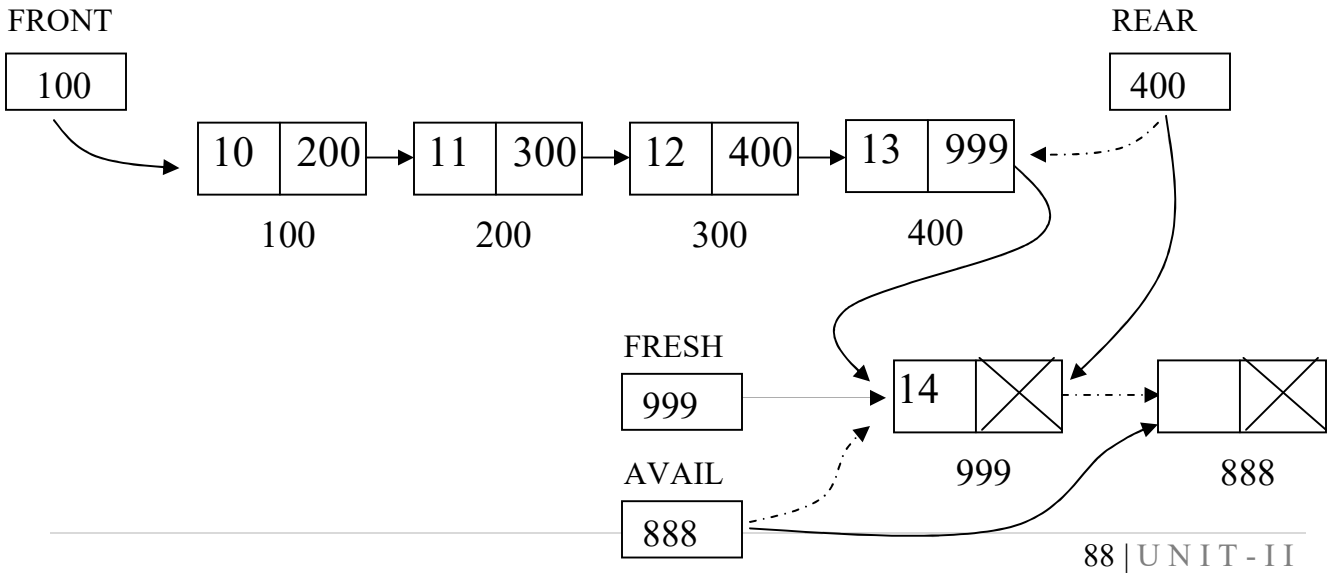
Step 1: If TOP=NULL, Then
Display "under flow"
Step 2: Else
PTR: =TOP
TOP: =LINK [TOP]
LINK [PTR]:=AVAIL
AVAIL: =PTR
[End of If]
Step 3: Exit

12.3 Linked Queue

When the concept of queue is implemented using singly linked list, it is referred as linked queue. In linked queue, assume FRONT is the pointer that identifies first node and REAR is the pointer that identifies the last node of linked queue.



12.3.1 Insertion operation in a linked queue:

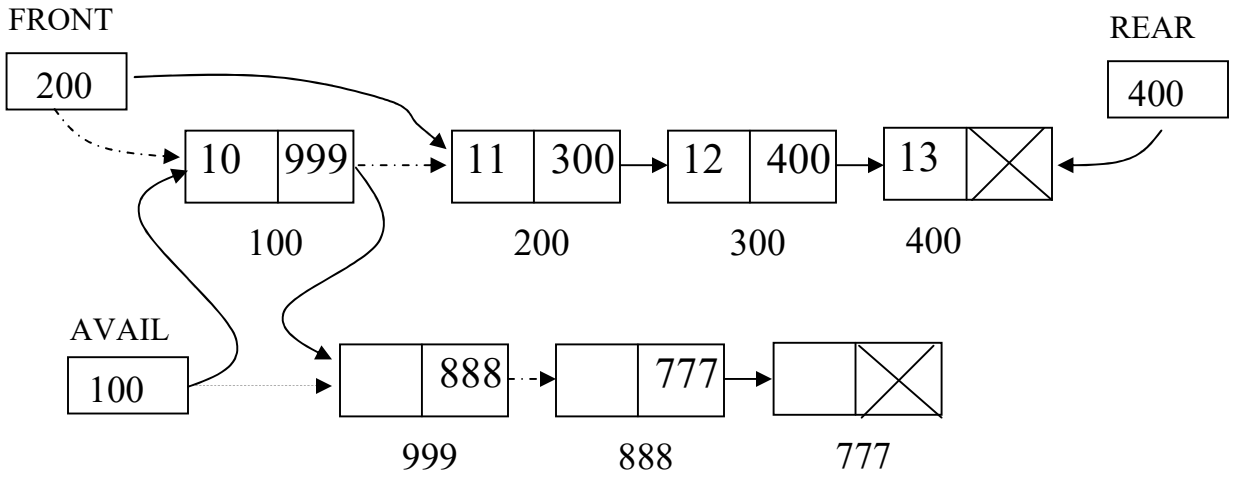


Algorithm:

LINKED_QUEUE_INSERTION (AVAIL, FRESH, LINK, FRONT, REAR, INFO, ITEM)

```
Step 1: If AVAIL =NULL, Then
            Display "Overflow"
Step 2: Else
            FRESH: =AVAIL
            AVAIL: =LINK [AVAIL]
            LINK [FRESH]:=NULL
            INFO [FRESH]:=ITEM
      Step 2.1: If FRONT=NULL AND REAR = NULL, Then
                  FRONT: =FRESH
                  REAR: =FRESH
      Step 2.2: Else
                  LINK [REAR]:=FRESH
                  REAR: =FRESH
              [End of If – Step 2.1]
      [End of If – Step 1]
Step 3:     Exit
```

12.3.2 Deletion operation in a linked queue:



Algorithm:

LINKED_QUEUE_DELETION (AVAIL, LINK, FRONT, REAR, PTR)

```
Step 1: If FRONT=NULL AND REAR=NULL, Then
            Display "Underflow"
Step 2: Else if     FRONT=REAR, Then
            FRONT: =NULL
            REAR: =NULL
Step 3:     Else
            PTR: =FRONT
            FRONT: =LINK [FRONT]
            LINK [PTR]:=AVAIL
            AVAIL: =PTR
        [End of If – Step 1]
Step 4: Exit
```

12.4 POLYNOMIAL REPRESENTATION USING LINKED LIST:

A single linked list can be used to represent and manipulate polynomials. A polynomial expression is a collection of polynomial terms, where each term contains a coefficient, base and exponent.

12.4.1 Representation of a polynomial term:

A node that can be used to represent a polynomial term is divided into 3 parts such as: Coefficient Part, Exponent Part and Link (or Address of next term in the polynomial expression) Part.

1. <u>ADDITION OF TWO POLYNOMIALS</u>
   Linked List are used to represent and manipulate polynomials.

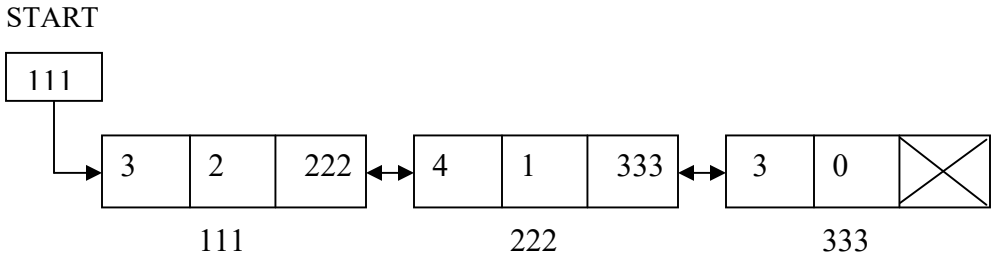
   e.g. Polynomials. $3x^2 + 4x\ 3$

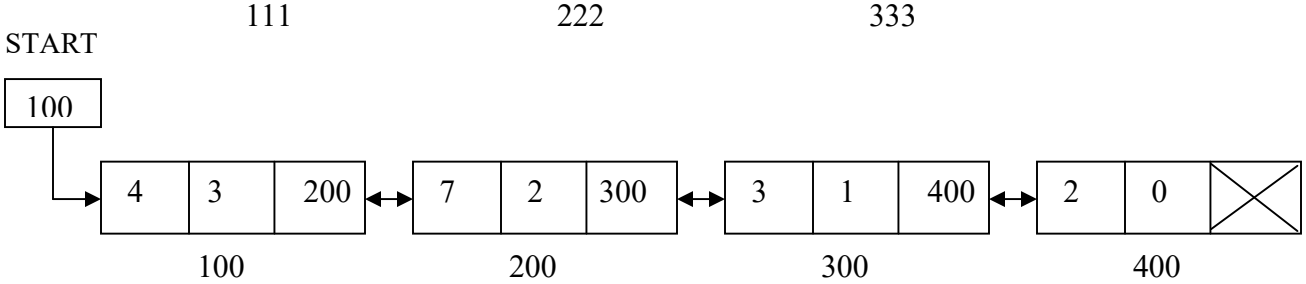
   Here in Polynomial representation in linked list each node having

   For Example : $3x^2$ can be represented as:

Co-efficient	Exponent	Link
3	2	200

1) $3x^2 + 4x^1 + 3x^0$
2) $4x^3 + 7x^2 + 3x^1 + 2x^0$

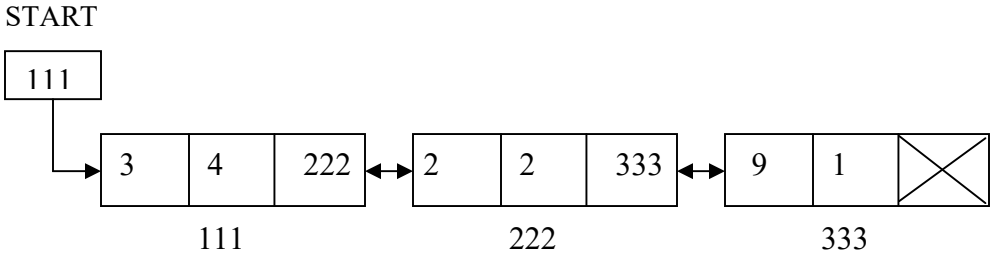


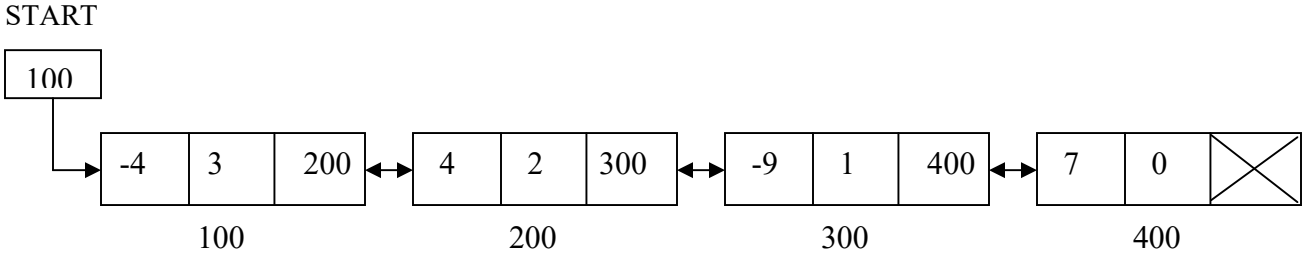


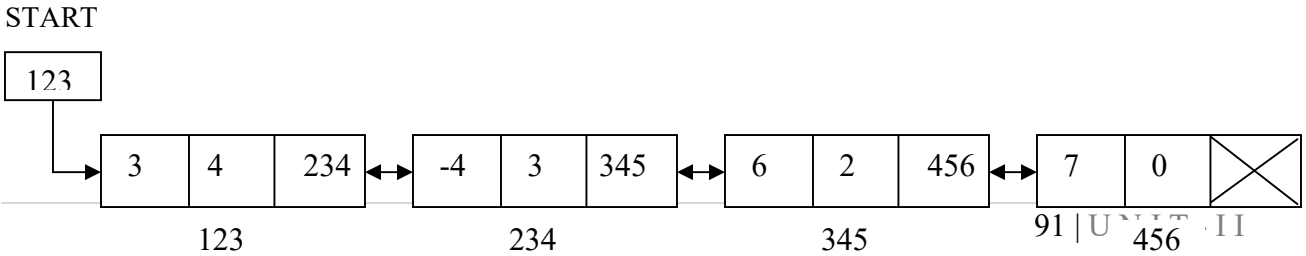
12.4.2 Polynomial Addition

P1 - $3x^4 + 2x^2 + 9x^1$
P2 - $-4x^3 + 4x^2 - 9x^1 + 7x^0$
Resultant P3 – $3x^4 + (-4x^3) + 6x^2 + 7x^0$

<u>Linked List Representations</u>







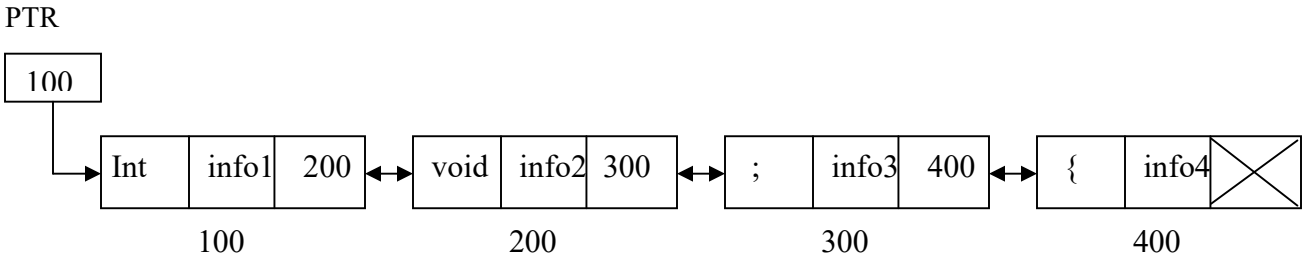
12.5 LINKED DICTIONARY

It is an important part of any compiler to construct and maintain a dictionary containing names and their associated values. Such dictionary is called as Symbol Table.

For example, in "C" Language, the linked dictionary (Symbol Table) contains identifiers, keywords, data types, special symbols, Operators, Constants etc.

Here their type of symbol is represented with their purpose i.e. associated values by the linked dictionary.

For Example:



There are two phases associated with construction of a symbol table

1. Building
2. Referencing

Building: Insertion of a now symbol along with its associated information in the end of linked dictionary.

Referencing: When it is desired to access a particular symbol, the table is searched sequentially from its beginning until it is found. So it will take n/2 comparisons at an average.

Here as the symbols are stored at the end in linked list dictionary. So insertion of a symbol is fast, but referencing is slow as it follows linear search.

12.6 DYNAMIC STORAGE MANAGEMENT PROCEDURES

12.6.1 Garbage collection

It is the process of collecting un-used nodes and returning them to available free space. There are two phases for it. A garbage collector is the one which

1. Marking all the nodes which are in use.
2. Return all unmarked nodes to the available free list.

12.6.2 Compaction

The problem with dynamic memory allocation is that the free memories are scattered with the actively used portions through out the memory.

When requests are for storage of fixed size, then it is enough to link all unused blocks, together.

When storage required of variable length then it might be insufficient.

During garbage collection, these memories are used further.

But as the memory is seriously fragmented, here the only way is to reallocate the used portions into one end of memory and combine all the holes i.e. free blocks into one large free area. This technique is called compaction (or) De fragmentation.

During compaction all running process must be stopped as data is copied from one memory area to another.

EXERCISES

LINKED LIST

1. How dynamic memory allocation differs from static memory allocation.
2. Differentiate between arrays and linked lists.
3. What do you mean by self referencing structure? Give one example.
4. What are the ways to make linked representation possible?
5. Explain the usage of malloc ( ) and free ( ) function.
6. Write short note on dynamic storage management.
7. Explain the overflow and underflow situation in a single linked list.
8. Write a program to implement a sorted linked list
9. Write a function that makes copies of the given linear linked list.
10. Write a 'C' program to reverse a singly linked list.
11. Write a 'C' code to search a node singly linked list based on given value.
12. Assuming info part of each node in a single linked list containing an integer value, write a 'C' code to find out sum of integers present each node.
13. Assume L1 and L2 are two single linked lists in the memory. Write a 'C' code to join them to create a single linked list L and also count total number of nodes in the resultant list.
14. Write a procedure to find length of a circular single linked list.
15. Write a function that concatenates two circular linked lists, producing a circular linked list.
16. Write a procedure to delete first node in a circular single linked list.
17. Write a procedure to insert a new node at the end of circular single linked list.
18. Differentiate between one-way list and two-way list.
19. What are the advantages and disadvantages of double linked list over single linked list?
20. Write a procedure to delete last node of double linked list. Explain with diagram.
21. Write a 'C' code to insert a new node at k^{th} position in circular double linked list.

APPLICATIONS OF LINKED LIST

22. Show how linked list can be used to represent the following polynomials
    $3x^4y - 5x^3y^3 - 7y + 3y^2$
    $7x^3y^2 + 8xy^2 - 3xy$
23. Write a 'C' function for evaluation of a polynomial.
24. List out the applications of linked list.

25. What is difference when stack implemented as linear array and stack implemented as linked list.
26. Write a ‘C’ code to perform push and pop operations on a linked stack.
27. What is linked queue? How does it differ from linear queue?
28. Explain garbage collection. Discuss advantage of garbage collection.
29. Explain how can a symbol table be constructed by using a linked list?
30. Write a short note on compaction.